

Chapter 1

General Context

Introduction

The real estate sector is currently undergoing a significant digital transformation, driven by the increasing need for operational efficiency, rapid communication, and centralized management. In an industry where timing and accurate information are critical, traditional management methods are no longer sufficient to meet the expectations of modern clients and agencies. Managing property listings, coordinating between different tiers of agents, handling client inquiries, and tracking visits require a robust, interconnected system. It is within this context that our end-of-studies project (Projet de Fin d'Études) is situated. This chapter serves as the foundation of the report, presenting the general context of the project. We will first introduce the host organization, followed by a critical study of the existing situation. Subsequently, we will define our proposed digital solution, detail the Agile SCRUM methodology adopted for its realization, and finally introduce the UML modeling language used for the system's design.

1.1 Host Organization

This section introduces the company that hosted this project, detailing its primary activities, values, and the services it offers to its clients.

1.1.1 Presentation of the Host Company

BESTTECH is a high-end IT services company, focused on quality, innovation, and speed. It uses information technologies to generate concrete results that help its clients' businesses advance. It operates in the fields of IT and digital services, technological solutions development, as well as training and technical skills enhancement, within an environment marked by technological innovation.

The main mission of BESTTECH is to support companies in their digital transformation by offering them high-performance, innovative solutions adapted to their needs, while respecting fundamental values such as ethics, integrity, and commitment to final results. Furthermore, BestTech collaborates with national and international companies, notably European ones, which allows it to open up to new markets, adopt international standards, and strengthen its expertise in a multicultural professional context.

Over the years, the company has successfully developed innovative technological solutions and established permanent assistance and support. This guarantees optimal reactivity, an effective response to specific client needs, the improvement of their digital presence, as well as a contribution to the development of technical skills through training.

1.1.2 Activities and Services Offered

BESTTECH provides a comprehensive suite of IT services and professional training:

1.1.2.1 IT Services

- **Web Development:** BESTTECH is committed to creating modular, scalable, and personalized websites closely tailored to client expectations. They offer showcase sites, E-business platforms, and dynamic web applications that are secure, highly performant, and compatible with major hosting platforms.
- **Mobile Development:** Mobile applications provide a highly promising access point for businesses, enabling the creation of new links with target audiences to increase visibility, brand awareness, and revenue.

- **Design:** Active participation in the creation of corporate visual identities, developing memorable brand imagery to attract new clients. This ensures a strong presence both online and offline, whether creating concepts from scratch or utilizing existing frameworks.
- **Search Engine Optimization (SEO):** BESTTECH enhances search engine rankings and site traffic. The expertise of its technical and content staff successfully meets the needs of businesses with complex websites in highly competitive sectors.
- **Community Management:** Social media experts manage communities to achieve established marketing objectives. Utilizing specific community management technological tools, they manage platforms strategically and qualitatively.
- **Consulting & Study:** Daily management experience of IT infrastructures guarantees a pragmatic approach to the projects managed. Consulting and support are at the core of the company, with IT security experts actively listening to clients and delivering comprehensive studies.

1.1.2.2 Professional Training

In addition to its IT services, BESTTECH offers several professional training programs available directly on its official website. These are aimed at students, recent graduates, and professionals wishing to develop their skills in information technologies. The training catalog covers:

- **Web and Mobile Development:**
 - *General Web Training:* A complete initiation curriculum covering development fundamentals (HTML, CSS), interface dynamization with JavaScript and jQuery, as well as server-side programming with PHP.
 - *Modern Frameworks & Technologies:* Specialization in top-tier tools highly demanded in Tunisian and international markets, including Angular, React.JS, Laravel (Advanced PHP), and Flutter for cross-platform mobile development.
- **Cloud Computing:** Curriculums dedicated to architecture and service management on Amazon Web Services (AWS).

- **Official Microsoft Azure Certifications:** A rich catalog including globally recognized certification paths such as Azure Fundamentals, Azure Administrator, Azure Solutions Architect, and Azure DevOps Engineer.
- **E-commerce and CMS:** Practical training for the design, deployment, and management of online stores via WordPress.

1.2 Study of the Existing Situation

Before proposing a new system, it is essential to analyze the existing methods used within the domain to understand their operational workflows, identify their shortcomings, and determine the necessary improvements.

1.2.1 Current Approach

Traditionally, many real estate agencies and independent agents rely on fragmented and manual approaches to manage their daily operations. The current workflow typically involves managing property portfolios through basic spreadsheets or standalone legacy software. Communication regarding inquiries, agent assignments, and property visits is often handled via disjointed channels such as phone calls, emails, and physical paperwork. Furthermore, media assets like property images are usually stored locally or manually uploaded without a centralized content delivery network, and inter-agent hierarchies (e.g., Head Agents overseeing Sub-Agents) are not formally structured within a unified digital platform.

1.2.2 Limitations and Challenges

The traditional approach presents several critical limitations that hinder productivity and growth:

- **Data Fragmentation and Inconsistency:** With information scattered across spreadsheets, emails, and paper documents, it is difficult to maintain a single source of truth, leading to frequent data entry errors and outdated property statuses.

- **Inefficient Task Management:** Manually assigning tasks, following up on client inquiries, and scheduling property visits consume a significant amount of administrative time that could otherwise be spent on core real estate activities.
- **Lack of Performance Visibility:** Without a centralized dashboard, administrators and Head Agents lack real-time insights into the performance, active listings, and client interactions of their Sub-Agents.
- **Absence of Automation:** Routine tasks, such as triggering notifications for new inquiries or updating property statuses upon sale approval, require manual intervention, resulting in slower response times and potential loss of leads.

1.2.3 Proposed Solution

To overcome the aforementioned challenges, this project aims to design and develop a comprehensive **Real Estate Automation Platform**. The proposed solution is a centralized, role-based web application tailored to streamline real estate operations. Key features of the proposed system include:

- **Role-Based Access Control (RBAC):** A strict 5-tier hierarchy providing distinct dashboards and permissions for Administrators, Head Agents, Sub-Agents, authenticated Clients, and unauthenticated Visitors, ensuring structured oversight and secure access to specific features.
- **Centralized Property and Inquiry Management:** A unified database to intelligently track listings, amenities, client favorites, and visit requests in real-time.
- **Workflow Automation:** Integration with automation tools (like n8n) to handle background processes, webhooks, and notifications automatically, reducing the burden of manual follow-ups.
- **Modern Technical Stack:** Leveraging cutting-edge technologies (such as Nuxt 3 for a dynamic frontend, FastAPI for a robust backend, Google Gemini 1.5 for AI interactions, and Ollama for vector embeddings) to provide a seamless, enterprise-grade user experience.

1.3 Agile Methodology

To ensure the successful development of this platform and to remain adaptable to evolving requirements, an agile approach was chosen to govern the project lifecycle.

1.3.1 Adopting SCRUM for Agile Project Development

For the realization of this project, we adopted the **SCRUM** framework. Unlike traditional predictable methodologies like the Waterfall model, SCRUM is an iterative and incremental agile software development framework. It was favored because it allows for continuous delivery of functional system components (such as individual dashboards or specific API integrations) while maintaining the flexibility to incorporate feedback and refine the UI/UX or backend business logic throughout the development process.

1.3.2 Identification of the Scrum Team

The success of a SCRUM project relies on the clear definition of roles. For this project, the team structure is defined as follows:

- **Product Owner (PO):** Represents the stakeholders or the host company. The PO defines the project requirements, prioritizes the Product Backlog, and ensures the end product delivers maximum value.
- **Scrum Master:** Responsible for ensuring that SCRUM practices are understood and followed, facilitating communication, and removing any impediments faced during development.
- **Development Team:** Responsible for the actual technical execution, encompassing both backend architecture and frontend interface design (in the context of an academic project, this role is primarily fulfilled by the student).

1.3.3 Scrum Events

The project workflow was organized around standard SCRUM ceremonies to maintain structure and momentum:

- **The Sprint:** The core of Scrum, consisting of time-boxed iterations (e.g., 2 to 3 weeks) during which a “Done”, usable, and potentially releasable product increment is created.
- **Sprint Planning:** A meeting at the beginning of each Sprint where the team selects items from the Product Backlog to work on and defines the Sprint Goal.
- **Daily Scrum (Stand-up):** A short periodic synchronization to plan work for the next 24 hours, track progress, and identify any blockers. (Often adapted in an academic context to regular progress syncs).
- **Sprint Review:** Held at the end of the Sprint to inspect the increment and adapt the Product Backlog if needed, involving demonstrations of completed features.
- **Sprint Retrospective:** An opportunity for the Scrum Team to inspect itself and create a plan for improvements to be enacted during the next Sprint.

1.3.4 Scrum Lifecycle

The lifecycle of our development process begins with the **Product Backlog**, a dynamic, prioritized list of all desired features (e.g., Authentication module, Sub-Agent Dashboard, Automation workflows). During Sprint Planning, the highest-priority items are moved into the **Sprint Backlog**. The Development Team then works on these items throughout the Sprint. At the end of the Sprint, the result is a functional **Product Increment** that is reviewed and validated against the initial requirements, before the cycle repeats for the next set of features.

1.4 Modeling language “UML”

To effectively translate the project requirements into a concrete technical architecture, we utilized **UML** (Unified Modeling Language). UML is a standardized visual modeling language

used to specify, visualize, construct, and document the artifacts of a software system.

Given the complexity of the Real Estate Automation Platform—which involves multiple user roles, robust database schemas, and intricate business logic mapping—UML provides an essential bridge between conceptualization and implementation. We employed various UML diagrams, such as:

- **Use Case Diagrams:** To map out the interactions between the different actors (Admin, Head Agent, Sub-Agent, Client, Visitor) and the system's functionalities.
- **Class Diagrams:** To define the static structure of the application, accurately modeling the database schema, entities (like Users, Properties, Inquiries), and their relationships.
- **Sequence Diagrams:** To detail the dynamic interactions and chronological sequence of messages between system components during critical processes (such as property assignment or handling a visit request).

Conclusion

In this introductory chapter, we have established the general context of the project. We analyzed the existing work environment, highlighting the limitations of traditional real estate management approaches, which led us to propose a modern, centralized Real Estate Automation Platform. Furthermore, we outlined the SCRUM Agile methodology that structures our development process and introduced the UML language chosen for our system modeling. With the foundational context and methodology established, the following chapter will delve into the specification and thorough analysis of the system's functional and non-functional requirements.

Chapter 2

Project Preparation

Introduction

Following our study of the existing processes and their limitations in the previous chapter, this second chapter is dedicated to the comprehensive preparation and meticulous planning of the **Elite Estate** Real Estate Automation Project. A successful software engineering endeavor, particularly one developed collaboratively within a constrained academic period, demands rigorous upstream organization. We will capture and detail both functional and non-functional requirements based precisely on our five distinct system roles. We will then outline the Agile SCRUM methodology adopted—expressly defining the detailed product backlog following standard industry templates, and the sprint planning schedules. Finally, we conclude by delineating the exhaustive hardware, software, and architectural choices that govern our technical implementation.

2.1 Requirement Capture

In this phase, we explicitly identify and categorize the precise needs of the system to ensure all stakeholder expectations are met.

2.1.1 Requirement Specifications

Defining formal specifications establishes the technical and functional foundation of the project. It explicitly denotes what the platform must do and how it must perform.

2.1.1.1 Functional Requirement Specifications

The functional requirements describe the specific behaviors and services the Elite Estate platform must deliver to its diverse users.

- **Authentication & Access Control:** Secure login and session management through robust JWT authentication.
- **Hierarchical User Management:** Admins create Head Agents and view platform-wide statistics. Head Agents allocate and manage Sub-Agents within their specific teams.
- **Property Management & Transactions:** Head Agents exclusively handle the creation and editing of property listings (uploading images via ImageKit) and approve transaction reports.
- **Visit Management & Team Calendar:** Sub-Agents manage and confirm assigned visit requests. Head Agents access a consolidated Team Calendar reflecting all appointments and scheduled visits.
- **Catalog Browsing & Web AI:** Unauthenticated Visitors navigate properties, view details, and query a generative RAG assistant directly on the website using pgvector embeddings.
- **Telegram Automations (Clients Only):** Authenticated Clients bypass traditional web dashboards entirely. They map their accounts to a Telegram Bot to inquire about properties, schedule visits directly, and receive automated n8n reminders before appointments.

2.1.1.2 Non-functional Requirement Specifications

Non-functional requirements describe criteria evaluating the overarching operation of the IT system:

- **Security:** API endpoints secured using JSON Web Tokens (JWT), passwords cryptographically hashed (pbkdf2_sha256).
- **Performance:** The FastAPI backend must handle real-time, asynchronous request processing competently with very low latency.
- **Interoperability:** The system must integrate seamlessly with external APIs (Google Gemini, Telegram, ImageKit, Ollama) through an ngrok-tunnelled n8n layer.

2.1.2 Modeling Requirements

To accurately model the system's interactions, it is crucial to first define the different types of users who will engage with the platform.

2.1.2.1 Identification of Actors

Based on our strictly defined Role-Based Access Control (RBAC), the actors are:

- **Admin:** Oversees platform configuration, views global statistics, and holds supreme user management (creating Head Agents).
- **Head Agent:** Acts as an organizational supervisor. They manage Sub-Agents, exclusively create/edit property listings, and oversee the unified Team Calendar.
- **Sub-Agent:** Performs daily operative behaviors focused entirely on managing assigned visit requests.
- **Client:** An authenticated user who strictly uses the Telegram Bot to schedule and track their visits.
- **Visitor:** An external, unauthenticated user discovering properties and interacting with the website's AI assistant.

- **AI Agent:** An autonomous system actor driven by generative AI models (Google Gemini 1.5) and vector databases. It processes natural language, evaluates calendar availability, recommends properties, and triggers automation tasks.

2.2 Global Use Case Diagram

The global use case diagram presented in Figure 2.1 illustrates the complete set of interactions between all system actors and the Elite Estate platform. It provides a bird's-eye view of every functional boundary, confirming that each actor interacts only with the use cases relevant to their hierarchical role.



Figure 2.1: Global Use Case Diagram – Elite Estate Platform

2.3 Project Management with Scrum

The adoption of SCRUM requires a structured team environment. In this section, we define the key roles responsible for driving the project forward.

2.3.1 Team and Roles

Our project strictly conforms to the SCRUM Agile framework ensuring rapid, flexible, and responsive iterations.

- **Product Owner (PO):** Outlines business requirements, dictates priorities, and establishes feature value.
- **Scrum Master:** Facilitates the development flow, enforces SCRUM ceremonies, and neutralizes blockers.
- **Development Team:** The executors assigned to code, design, and architect the platform.

2.3.2 Product Backlog

The Product Backlog is the central point of any Scrum project and serves as the primary input for sprint planning. It consists of a list of product items prioritized based on importance and complexity.

Below is an explanation of the fields used in the Product Backlog:

- **ID:** A unique number for each feature.
- **Feature:** Categorizes the product item under a broader functionality.
- **PI/US ID:** Unique identifier for each product item (PI) or User Story (US).
- **Product Item / User Story:** A description of the functional requirement, often following the narrative format: *"As a [role], I want to [goal]."*
- **Priority:** Indicates the importance or urgency.

- **Complexity:** Refers to the effort involved using the Fibonacci sequence (1, 2, 3, 5, 8, 13, 21).

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
1	Feature Configuration	PI1.1	Configure the multi-container Docker environment (FastAPI, Nuxt, PostgreSQL, Ngrok) so that all platform services run under a unified local network.	High	13
		PI1.2	Set up and configure the self-hosted n8n platform environment and secure webhook tunnels.	High	8
		PI1.3	Design and implement the TAIA Smart Agent and Meeting Reminder workflows in the n8n UI.	High	13
2	User Auth	US2.1	As a Visitor, I want to securely register an account so that I can become a Client.	High	13
		US2.2	As a Visitor / Client, I want to log in using my credentials to access my specific features.	High	8
		US2.3	As a Sub-Agent / Head-Agent / Admin, I want to log in securely so that I can access my operational dashboard.	High	8
		US2.4	As a Client / Sub-Agent / Head-Agent / Admin, I want to securely log out so that my session is protected.	High	5

Continued on next page

Table 2.1 – *Continued from previous page*

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
3	Platform Hierarchy	US3.1	As an Admin, I want to view all registered users so that I can monitor the user base.	High	8
		US3.2	As an Admin, I want to create and authorize Head-Agent accounts so that they can manage properties.	High	13
		US3.3	As an Admin, I want to create Sub-Agent accounts and assign them to specific Head-Agents so that teams are structured.	High	13
		US3.4	As an Admin, I want to enable or disable any user account so that I can moderate platform access without deleting data.	High	8
		US3.5	As a Head-Agent, I want to enable or disable my assigned Sub-Agent accounts so that I can manage my staff.	Medium	8
3	Platform Hierarchy	US3.6	As a Client / Sub-Agent / Head-Agent / Admin, I want to view and update my basic profile information so that my contact details are current.	Medium	13
		US3.7	As an Admin, I want to search and filter users by role so that I can manage accounts efficiently.	High	8

Continued on next page

Table 2.1 – *Continued from previous page*

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
4	Property Management	US4.1	As a Head-Agent, I want to add new properties with detailed specs so that they are available in the catalog.	High	13
		US4.2	As a Head-Agent, I want to upload high-resolution image galleries for properties so that they look appealing.	High	13
		US4.3	As a Head-Agent, I want to edit existing property details and prices so that the information stays accurate.	High	13
		US4.4	As a Head-Agent, I want to delete outdated property listings so that the catalog remains relevant.	Medium	8
		US4.5	As a Head-Agent, I want to filter and search through my own property listings so that I can manage them efficiently.	Medium	8
		US4.6	As a Sub-Agent, I want to consult, search, and filter my assigned properties so that I can manage them effectively.	High	8

Continued on next page

Table 2.1 – *Continued from previous page*

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
5	Catalog Discovery	US5.1	As a Visitor / Client, I want to browse the full list of available properties with pagination so that I can discover homes.	High	8
		US5.2	As a Visitor / Client, I want to filter properties by price, area, and amenities so that I can find exactly what I need.	High	13
		US5.3	As a Visitor / Client, I want to view comprehensive property details and images so that I can make an informed decision.	High	8
		US5.4	As a Visitor / Client, I want to view the geographical location of a property via interactive Leaflet maps so that I know the neighborhood.	High	13
6	AI Intelligence	US6.1	As a Visitor / Client, I want the AI to retrieve accurate property context (RAG) before answering so that I can receive truthful, contextual information.	High	21
		US6.2	As a Visitor / Client, I want the AI to perform semantic vector searches so that I can discover and get recommendations for highly similar listings.	High	21

Continued on next page

Table 2.1 – *Continued from previous page*

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
7	TAIA	US7.1	As a Client, I want to link my Telegram account via a pairing code so that TAIA can identify me and send personalized notifications.	High	13
		US7.2	As a Visitor / Client, I want the AI to parse reschedule or cancellation requests and update the appointment record instantly so that appointment records are synchronized.	High	13
8	Visit Management	US8.1	As a Sub-Agent, I want new TAIA-booked visits to automatically appear on my dashboard as “Scheduled”.	High	13
		US8.2	As a Sub-Agent, I want to manually change the status of an appointment to “Finished” or “Cancelled” from my dashboard.	High	8
		US8.3	As a Sub-Agent, I want to filter my assigned visits so that I can organize my daily schedule efficiently.	Medium	8
9	Notifications	US9.1	As a Client / Sub-Agent, I want to receive automated reminder notifications via TAIA and Email prior to our scheduled meeting.	High	21

Continued on next page

Table 2.1 – *Continued from previous page*

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
		US9.2	As a Client / Sub-Agent / Head-Agent / Admin, I want to receive automated emails for my specific transaction operations (e.g., Client/Admin receives reports, Sub/Head-Agent receives responses).	High	13
10	Analytics & Oversight	US10.1	As a Head-Agent, I want to view a team calendar in my dashboard displaying all appointments of my assigned Sub-Agents with statuses and filters.	High	13
		US10.2	As a Head-Agent, I want to consult my personal appointment calendar in the dashboard so that I can track my own schedule.	High	13
		US10.3	As a Head-Agent, I want to view my analytics statistics and team performance so that I can evaluate my operations.	High	13
		US10.4	As an Admin, I want to view an analytics dashboard of platform growth and total transactions to measure global success.	High	13

Continued on next page

Table 2.1 – *Continued from previous page*

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
		US10.5	As a Sub-Agent, I want to view my personal analytics based on completed visits so that I can track my performance.	Medium	8
		US10.6	As an Admin or Head-Agent, I want to download detailed transaction reports so that I can keep external records of platform deals.	High	13
11	Finalization	US11.1	As a Sub-Agent, I want to submit transaction requests and complete them once approved (Request Sale/Rent and Complete Transaction).	High	13
		US11.2	As a Head-Agent, I want to approve a transaction request, allowing the Sub-Agent to proceed with completion.	High	13
		US11.3	As a Head-Agent, I want to refuse a transaction request, which will notify the Sub-Agent via Email of the refusal.	Medium	8
		US11.4	As a Head-Agent, I want to consult my rented and sold properties so that I can track my historically finalized deals easily.	Medium	8
Total Complexity Points					490

Table 2.1: Product Backlog

Sprint 1 — Setup, User Authentication & Platform Hierarchy (118 points)

This sprint establishes the foundational environment and development infrastructure. It covers the multi-container Docker setup, the configuration of the two n8n automation workflows (TAIA Smart Agent and Meeting Reminder Service), user authentication (registration, login, logout), and core team hierarchies (creating Head-Agents and active Sub-Agents).

Sprint 2 — Property Management, Catalog Discovery & Profile Management (126 points)

This sprint covers the full lifecycle of real estate properties, catalog discovery with pagination/filters, interactive maps, alongside profile updates and user search features.

Sprint 3 — AI Intelligence, TAIA & Visit Management (97 points)

This sprint implements the AI-powered core of the platform (Gemini and Ollama semantic search), TAIA conversational bot, visit scheduling, Sub-Agent personal analytics views, and transaction report generations.

Sprint 4 — Notifications, Advanced Analytics & Finalization (149 points)

The final sprint delivers automated oversight, email/TAIA reminders, team and personal calendar views, Head-Agent/Admin dashboards, and transaction finalization approval chains.

2.3.3 Detailed Sprint Planning and Resource Allocation Table

Below is a detailed Sprint Planning table that maps each Product Item (PI) to its concrete implementation pages or backend files, along with the specific development hours allocated for each page to be completed:

Sprint	PI/US	Product Item	Tasks	h
	PI1.1	Docker environment setup	Create <code>docker-compose.yml</code> , configure FastAPI and Nuxt Dockerfiles for unified networking	10

Continued on next page

Sprint 1

Setup, Auth

Table 2.2 – *Continued from previous page*

Sprint	PI/US	Product Item	Tasks	h
	PI1.2	n8n environment setup	Set up and configure the self-hosted n8n platform environment and secure webhook tunnels	6
	PI1.3	n8n workflows configuration	Design and implement the TAIA Smart Agent and Meeting Reminder workflows in the n8n UI	12
	US2.1	Account registration	Implement registration UI, integrate <code>useAuth</code> composable, build backend auth route	12
	US2.2	Client / Visitor login	Build login UI, implement JWT token storage and state management	8
	US2.3	Staff / Admin login	Add role-based redirection post-login, secure dashboard routes	8
	US2.4	Secure logout	Implement logout handler, clear local storage tokens	2
	US3.1	View registered users	Build Admin Users tab UI, implement <code>useAdminUsers</code> composable	8
	US3.2	Create Head-Agent	Build Head-Agent creation modal, connect to backend auth endpoint	10

Continued on next page

Table 2.2 – Continued from previous page

Sprint	PI/US	Product Item	Tasks	h
	US3.3	Create Sub-Agent	Build Sub-Agent creation modal with Head-Agent assignment logic	10
	US3.4	Toggle account status	Implement Admin action to enable/disable any user account	6
	US3.5	Toggle Sub-Agents	Add Agency Staff tab, implement enable/disable toggles for Sub-Agents	6
Sprint 2 Properties & Catalog	US3.6	Edit profile details	Build unified profile page, connect update endpoints	8
	US3.7	Search and filter users	Implement search bar and role filters in Admin Users tab	5
	US4.1	Add properties (CRUD)	Build Agency Properties tab, create property submission form	16
	US4.2	ImageKit gallery upload	Integrate ImageKit CDN, build multi-image upload component	10
	US4.3	Edit property specs	Build property edit modal, handle partial updates to API	8
	US4.4	Delete outdated listings	Implement soft-delete logic for Agency properties	4
	US4.5	Filter agent listings	Add status and location filters to Agency Properties tab	5

Continued on next page

Table 2.2 – Continued from previous page

Sprint	PI/US	Product Item	Tasks	h
	US4.6	Sub-Agent assigned view	Build Agent Properties tab, display assigned listings	5
	US5.1	Browse catalog	Implement global properties grid, integrate pagination logic	14
	US5.2	Catalog filtering	Build advanced filter sidebar (price, area, amenities)	14
	US5.3	Property details page	Design dynamic <code>[slug].vue</code> page, display full specifications	10
	US5.4	Leaflet interactive maps	Integrate Nuxt Leaflet, display property coordinates on map	12
Sprint 3 AI & TAIA	US6.1	RAG context retrieval	Implement LangChain pipeline, connect Gemini 1.5 Pro to PGVector	18
	US6.2	Semantic vector search	Set up Ollama embeddings, implement cosine similarity search	18
	US7.1	Telegram pairing	Implement pairing code generation, build Telegram bot handler	12
	US7.2	AI cancel / reschedule	Parse intents via n8n, dynamically update Visit records	14
	US8.1	Sync TAIA bookings	Build Agent Visits tab, fetch and display scheduled visits	10

Continued on next page

Table 2.2 – Continued from previous page

Sprint	PI/US	Product Item	Tasks	h
	US8.2	Update appointment status	Add action buttons to change visit status (Finished/Cancelled)	8
	US8.3	Filter agent visits	Implement date and status filters on Agent Visits tab	5
Sprint 4 Notifications & Deals	US9.1	TAIA & email reminders	Configure n8n cron node, format reminder email templates	20
	US9.2	Transaction emails	Set up SMTP backend triggers for transaction state changes	8
	US10.1	Head-Agent team calendar	Implement full-calendar view in Agency dashboard, fetch team visits	16
	US10.2	Head-Agent personal calendar	Add personal visit filter to Agency calendar view	12
	US10.3	Head-Agent analytics dashboard	Build Agency Analytics tab, compute team conversion rates	14
	US10.4	Admin global analytics	Build Admin Analytics tab, aggregate platform growth metrics	12
	US10.5	Sub-Agent personal analytics	Build Agent Analytics tab, calculate personal success metrics	10
	US10.6	Download reports PDF	Implement ReportLab generation in FastAPI, add download endpoints	14

Continued on next page

Table 2.2 – *Continued from previous page*

Sprint	PI/US	Product Item	Tasks	h
	US11.1	Submit & complete transaction	Add action buttons for Sale/Rent requests and final completion	14
	US11.2	Approve transaction	Build Agency Requests tab, implement approval endpoint logic	10
	US11.3	Refuse transaction	Add refusal flow, require reasoning, trigger notification	10
	US11.4	Track sold / rented history	Build historical records view in Agency Properties tab	5
Total Development Hours				439 h

Table 2.2: Detailed Sprint Planning and Resource Allocation

2.4 Work Environment

Developing a full-stack automation platform requires a robust environment. This section details the physical and software tools utilized during the project’s lifecycle.

2.4.1 Hardware Environment

The development, local compilation, container orchestration, and AI model execution were distributed and performed across two distinct development workstations to ensure high productivity, hardware redundancy, and secure resource isolation. The detailed hardware configurations of these two machines are summarized in Table 2.3:

Attribute	Primary Workstation (PC 1)	Secondary Workstation (PC 2)
Processor (CPU)	Intel Core i7 12th Generation	Intel Core i5 11th Generation
Memory (RAM)	24 GB DDR4	16 GB DDR4
Storage Capacity	512 GB NVMe SSD	512 GB NVMe SSD
Operating System	Windows 11	Windows 11

Table 2.3: Hardware Specifications of the Development Workstations

2.4.2 Software Environment

A sophisticated stack of programming languages, libraries, modeling platforms, and communication services was assembled to construct, virtualize, type-set, and coordinate this platform. All utilized software components are listed independently in Table 2.4, with dedicated vertical spacing left in the first column to accommodate their official logos.

Software (Logo)	Description
	Nuxt 3 & Vue 3: Used as the core responsive web frontend framework. It leverages Server-Side Rendering (SSR) for initial load speed and the Composition API to establish modular, reactive real estate dashboards.
	FastAPI (Python): A modern, high-performance asynchronous microframework (ASGI) powering the backend API, executing tasks concurrently and managing RAG vector operations with zero speed bottlenecks.
	PostgreSQL & pgvector: The primary ACID-compliant relational database, extending its standard capabilities through pgvector to persist and query 768-dimensional conceptual coordinates.

Continued on next page

Table 2.4 – Continued from previous page

Software (Logo)	Description
	n8n Orchestrator: The self-hosted digital automation server coordinating webhook data, managing account scheduling background tasks, and synchronizing workflows securely.
	Google Gemini 1.5: The state-of-the-art multi-modal LLM driving the cognitive reasoning and RAG conversational dialogue loops with the client.
	Ollama (nomic-embed-text): Deployed locally to convert property descriptions and user queries into conceptual embeddings vector arrays completely offline.
	ImageKit SDK: The integrated Content Delivery Network (CDN) utilized for compressing, optimizing, and serving property listing media galleries instantly.
	Telegram Bot API: The platform used by clients to communicate directly with the TAIA assistant, allowing visit scheduling and account pairing over mobile and desktop.
	TypeScript: The primary language used for frontend development, extending JavaScript with static types to intercept and resolve run-time errors before build.

Continued on next page

Table 2.4 – *Continued from previous page*

Software (Logo)	Description
	Python (3.11): The backend programming language hosting FastAPI routes, data schema validation, and interfacing naturally with the local AI models.
	SQL: Formulated directly and through ORMs to define databases, establish role hierarchies, and construct complex join queries securely.
	Docker & Compose: Used for isolating and containerizing each component (API, web, db, ngrok, n8n) into unified local networking clusters matching production state.
	Git & GitHub: The distributed version control workflow utilized to track code modifications, merge features, and secure files collaboratively.
	Visual Studio Code: The primary Integrated Development Environment (IDE) equipped with customized workspaces, linters, and compilers for Python and Vue.
	TeXstudio & LaTeX: The professional writing, typesetting, and rendering suite utilized to compile the architectural designs and documentation of this report.

Continued on next page

Table 2.4 – *Continued from previous page*

Software (Logo)	Description
	Draw.io: The diagramming engine utilized to draft the structured UML diagrams, system deployment schemes, and application logical views.
	Google Meet: The collaboration application used to host synchronization meetings, present project checkpoints, and review code deliverables.
	Telegram Messenger: Deployed on mobile and desktop workstations to verify real-time chat responsiveness and pairing flows with the active bot.

Table 2.4: Software Technologies, Frameworks, and Tools

2.5 Architecture

The architectural design forms the blueprint of the platform. We establish both the physical deployment infrastructure and the logical application layers.

2.5.1 System Architecture

We established a container-centric physical infrastructure. The core elements operate across distinct, communicative dockerized containers coordinated through a `docker-compose.yml` network context. This ecosystem cleanly segregates the Database tier away from the Application Processing node and identically isolates the active Automation Server. We additionally integrated an ngrok sidecar container to bridge HTTPS webhooks for Telegram testing.

Diagram Implementation Note

[Note for Author: Please generate and insert the System Architecture (Deployment) Diagram below. You can replace the placeholder space with the actual image file.]

[Placeholder: Insert your System Architecture (Deployment) Diagram here]

Figure 2.2: System Deployment Architecture

2.5.2 Application Architecture

On a logical basis, the platform is designed and structured following a robust **N-Tier (Multi-Tier) Architecture** that guarantees solid segregation of concerns, horizontal scalability, and secure decoupled deployment:

- **Presentation Tier (Frontend):** Implemented using Nuxt 3 and Vue 3. It handles the visual layout, user interactions, and state management (via Pinia) entirely inside the client browser, securing a rich, responsive experience.
- **Business / Application Logic Tier (Backend):** Hosted by FastAPI. This tier encapsulates all the core business logic, asynchronous task queues, authorization guards (JWT), and handles system integrations (Google Gemini, Ollama, and n8n webhooks).
- **Data Tier (Persistence):** Powered by PostgreSQL equipped with the pgvector extension. It represents the data layer managing the relational schema (Users, Properties, Visits) and vector embeddings securely, orchestrated through the SQLAlchemy ORM layer.

Diagram Implementation Note

[Note for Author: Please generate and insert the Application Architecture Diagram (Frontend/Backend) below.]

[Placeholder: Insert your Application Architecture Diagram (Frontend/Backend) here]

Figure 2.3: Application Architecture

Conclusion

A detailed analysis and well-defined requirements provide a strong base for the development of the Elite Estate Platform. The technical specifications and architectural design outlined in the project will serve as a roadmap for the design and implementation stages, ensuring the platform is built efficiently and meets both functional and non-functional expectations.

Chapter 3

State of the Art and Technological Choices

Introduction

Before beginning the design and implementation of the Real Estate Automation Platform, it is crucial to conduct a comprehensive study of the existing technologies in the market. The core challenge of this project lies in designing a hybrid architecture that seamlessly combines a Conversational AI capable of natural language understanding, a Workflow Orchestration Engine for automated coordination, and a robust Web Development Stack. This chapter presents a rigorous state-of-the-art bibliographic study comparing various tools and frameworks. Based on rigorous evaluation criteria—including cost, performance, developer velocity, and AI synergy—we will utilize comparative matrices to justify the final technological stack chosen to fulfill the project’s requirements.

3.1 The AI Conversational Assistant

The evolution of Natural Language Processing (NLP) has shifted dramatically from rigid, rule-based AI assistants to dynamic Large Language Models (LLMs). For our real estate platform, we built TAIA (Telegram AI Assistant), a conversational interface capable of extracting temporal entities from client messages, understanding search intent, and negotiating

visit availability based on dynamic constraints.

3.1.1 Large Language Models (LLMs)

To select the optimal engine for the conversational assistant, we evaluated three leading model families based on reasoning capability, latency, and cost.

Model Family	Reasoning & Context	Speed (Latency)	Cost Profile	Verdict
Google Gemini 1.5 (Flash/Pro)	Exceptional multi-modal reasoning. Massive context window (1M+ tokens).	Ultra-fast (Flash variant)	Extremely cost-efficient	SELECTED
OpenAI GPT-4o	Industry benchmark for complex reasoning and function calling.	Moderate	High cost at scale	Rejected
Anthropic Claude 3.5	Excellent context retention and coding capabilities.	Fast	Medium-High	Rejected

Table 3.1: Comparative Matrix of Large Language Models (LLMs)

Google Gemini 1.5 was selected as the primary model. It offers an unparalleled balance of speed — crucial for TAIA real-time responses — and advanced reasoning capabilities necessary for scheduling constraints. The Flash variant is optimized for instantaneous chat responses, while the Pro variant handles deep property logic seamlessly.

3.1.2 AI Semantic Search & Embedding Models

Traditional search relies on keyword matching — searching for “House with a pool” might miss “Villa with a swimming area”. To solve this, we implemented AI Semantic Search to evaluate the conceptual intent of user queries rather than exact lexical matches.

Semantic search evaluates the meaning of a query rather than specific words. Imagine every idea in the world is a location on a giant map. “House” and “Villa” are parked right next to each other. To achieve this, an embedding model acts as a translator. When a property

is added, the model converts its description into a dense vector — a list of 768 numbers representing the property's semantic signature. When a user searches, their query is mapped the same way, and the database retrieves the closest matching properties.

To implement this semantic search, we deployed the `nomic-embed-text` model locally via Ollama. This high-performance model simplifies text translation into mathematical coordinates through a straightforward, three-step pipeline:

1. **Tokenization (Breaking Down the Text):** The raw input text (e.g., "modern apartment near downtown") is split into small pieces called tokens (words or sub-words). Each token is assigned a unique numerical ID from a pre-defined vocabulary, transforming plain text into a structured list of numbers.
2. **Context Analysis (Understanding Meaning):** The model evaluates how these tokens relate to one another in both directions. This step ensures that the system understands the actual context — for example, distinguishing a "villa with a pool" (swimming facility) from a "pooling agreement" (financial term).
3. **Vector Compilation (The Semantic Map):** Finally, all token-level meanings are averaged and compressed into a single, static array of 768 decimal numbers (an embedding vector). This vector represents the unique "semantic signature" or coordinate of the property in a multi-dimensional conceptual space.

Vector storage and query matching via `pgvector`:

Once this 768-dimensional array is generated, the FastAPI backend stores it in PostgreSQL in a column typed as `vector(768)` using the `pgvector` extension.

When a user searches the catalog using everyday language (e.g., "cozy house near the sea"), the platform converts their search query into a 768-dimensional query vector using Ollama. The database then performs a search by calculating the geometric distance between the query vector and all the property vectors in the database.

This distance (specifically the Euclidean or L2 distance) represents how conceptually similar the listings are:

$$\text{Distance} = \sqrt{\sum (\text{Query Component} - \text{Property Component})^2} \quad (3.1)$$

A smaller distance means the coordinates are closer together on our semantic map, indicating a higher conceptual match. The system instantly ranks and retrieves the top 5 most similar listings, ensuring extremely fast, cost-free, and highly accurate results without needing external cloud APIs.

3.1.3 Retrieval-Augmented Generation (RAG)

While LLMs provide exceptional reasoning capabilities, they suffer from knowledge staleness. A standard AI model possesses no knowledge of a new villa listed yesterday or an agent's current availability.

Without external knowledge, an AI model may hallucinate incorrect answers. Retrieval-Augmented Generation (RAG) solves this by acting as an open-book exam. Instead of relying on its parametric memory, the system queries the vector database to retrieve accurate, up-to-date property metadata before formulating an answer. The AI is instructed to respond *only* using the retrieved context, ensuring factual accuracy and personalization across all TALA conversations.

3.2 Workflow Orchestration

While the AI handles the cognitive processing, the Orchestration Layer acts as the nervous system, triggering APIs, managing state, and handling errors. We compared several platforms to find the optimal balance between real-time responsiveness and robust logic.

3.2.1 Comparative Analysis of Orchestration Platforms

Platform	Cost / Hosting	Latency	Flexibility	Verdict
n8n	Free (Self-hostable)	Fast (Webhook)	High (Code Nodes)	SELECTED
Zapier	SaaS (High Cost)	Fast	Limited	Rejected
Make	SaaS (Medium Cost)	Fast	Good	Rejected
Airflow	High Infra Cost	Slow (Batching)	Very High (Python)	Rejected

Table 3.2: Orchestration Engine Selection Matrix

n8n was selected as the central automation engine. The platform’s services — database, AI models, and Telegram — cannot communicate natively; n8n bridges them through two dedicated workflows.

The first, the TAIA Smart Agent workflow (`Elite Estate - Smart Agent service.js`) manages the full conversational cycle: it receives client messages from TAIA, routes them to Google Gemini with the RAG context retrieved from the database, and returns structured responses.

The second, the Meeting Reminder Service workflow (`Elite Estate - Meeting Reminder Service (7).json`), runs on a scheduled trigger. It queries the database for upcoming appointments within the next hour and automatically dispatches reminder notifications to Clients and Sub-Agents via TAIA and email.

3.3 Software Development Stack

The development stack constitutes the "body" of the application, encompassing the database, the backend business logic, and the visible frontend web dashboard.

3.3.1 Database Persistence

Real estate data is inherently relational (Clients possess appointments; appointments belong to properties; properties are managed by agents). We evaluated three major database architectures.

Database	Type	Schema Enforcement	AI Vector Support	Verdict
PostgreSQL	Relational	Strict (ACID Compliant)	Excellent (pgvector extension)	SELECTED
MongoDB	NoSQL	Flexible (Unstructured)	Native but non-relational	Rejected
MySQL	Relational	Strict	Limited native vector tools	Rejected

Table 3.3: Database Architecture Comparison

Technological Justification: **PostgreSQL** was selected because strict ACID compliance (Atomicity, Consistency, Isolation, Durability) is mandatory to prevent double-booking real estate visits. Furthermore, the **pgvector** extension allows PostgreSQL to store the 768-dimensional Ollama embeddings, directly integrating our semantic search capabilities into the relational core without needing a separate vector database (like Pinecone).

3.3.2 Frontend Frameworks

The frontend interface must handle high-quality media, complex booking filters, and provide an excellent user experience.

Framework	Language	Rendering	Learning Curve	Verdict
Nuxt 3 (Vue)	TypeScript	Native SSR / SSG	Gentle, highly intuitive reactivity	SELECTED
Next.js (React)	TypeScript	SSR / Server Components	Steep (RSC hydration complexities)	Rejected
Angular	TypeScript	CSR / Universal	Very Steep (Heavy boilerplate)	Rejected

Table 3.4: Frontend Framework Comparison

Technological Justification: **Nuxt 3 (Vue) powered by TypeScript** was selected. It provides native Server-Side Rendering (SSR) for critical SEO discoverability, while component auto-imports and intuitive state management (Pinia) guarantee incredibly high development velocity, aligning perfectly with the project’s timeline constraints.

3.3.3 Backend Frameworks

The backend serves the frontend and handles complex logic that surpasses visual workflow engines, interfacing directly with PostgreSQL and the AI models.

Framework	Language	Concurrency	AI Ecosystem Integration	Verdict
FastAPI	Python	Native async/await (ASGI)	Flawless (Native to Data Science)	SELECTED
Express.js	Node.js	Non-blocking I/O	Cumbersome AI wrappers required	Rejected
Django	Python	Synchronous (WSGI standard)	Good, but heavy framework	Rejected

Table 3.5: Backend Framework Comparison

Technological Justification: Python (FastAPI) was the definitive choice for the server. Python is the undisputed native language of Artificial Intelligence. Using FastAPI eliminates the friction of translating data between AI models and the server natively, while its ASGI architecture provides high-performance asynchronous request handling comparable to Node.js.

Conclusion

This bibliographic study established the definitive technological foundation for the Real Estate Automation Platform. Through rigorous comparative analysis via evaluation matrices, we determined that combining **Google Gemini 1.5** with an **Ollama-powered Semantic Search** architecture ensures the highest conversational accuracy and intelligence. For orchestration, **n8n** offered the optimal balance as a Digital Orchestrator providing real-time speed and flexibility. For the software architecture, **PostgreSQL (with pgvector)** guarantees data integrity, **Nuxt 3 with TypeScript** ensures rapid, type-safe frontend development, and **Python (FastAPI)** provides the vital backend synergy necessary to host complex AI operations seamlessly. With the state of the art thoroughly explored and the final stack justified, the following chapters will detail the system’s design and physical implementation.

Chapter 4

Sprint 1 - User Authentication & Platform Hierarchy

Introduction

Sprint 1 establishes the foundational infrastructure and access control layer of the Elite Estate platform. It covers the configuration of the multi-container Docker environment, the setup of the two n8n automation workflows, and the implementation of all user authentication mechanisms. It also delivers the full administrative hierarchy, enabling the Admin to create and manage Head-Agents and Sub-Agents with structured role-based permissions.

4.1 Requirement Specifications

In this section, we present the Sprint 1 backlog, the corresponding use case diagram, and the complete textual descriptions of all use cases addressed during this sprint.

4.1.1 Sprint Backlog

The following product items were selected for Sprint 1, accompanied by their priority and complexity:

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
1	Feature Config.	PI1.1	Configure the multi-container Docker environment (FastAPI, Nuxt, PostgreSQL, Ngrok) so that all platform services run under a unified local network.	High	13
		PI1.2	Set up and configure the two self-hosted n8n workflows (TAIA Smart Agent and Meeting Reminder Service) with webhook tunnels to support asynchronous automation processes.	High	21
2	User Auth	US2.1	As a Visitor, I want to securely register an account so that I can become a Client.	High	13
		US2.2	As a Visitor / Client, I want to log in using my credentials to access my specific features.	High	8
		US2.3	As a Sub-Agent / Head-Agent / Admin, I want to log in securely so that I can access my operational dashboard.	High	8
		US2.4	As a Client / Sub-Agent / Head-Agent / Admin, I want to securely log out so that my session is protected.	High	5

Continued on next page

Table 4.1 – Continued from previous page

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
3	Platform Hierarchy	US3.1	As an Admin, I want to view all registered users so that I can monitor the user base.	High	8
		US3.2	As an Admin, I want to create and authorize Head-Agent accounts so that they can manage properties.	High	13
		US3.3	As an Admin, I want to create Sub-Agent accounts and assign them to specific Head-Agents so that teams are structured.	High	13
		US3.4	As an Admin, I want to enable or disable any user account so that I can moderate platform access without deleting data.	High	8
		US3.5	As a Head-Agent, I want to enable or disable my assigned Sub-Agent accounts so that I can manage my staff.	Medium	8

Table 4.1: Sprint 1 Backlog

4.1.2 Use Case Diagram

The use case diagram below illustrates the interactions addressed in Sprint 1, specifically covering account registration, authentication, and the administrative hierarchy management

capabilities of the Admin and Head-Agent actors.



Figure 4.1: Use Case Diagram of Sprint 1

4.1.3 Class Diagram

The class diagram below models the static entities introduced in Sprint 1. It highlights the User entity and its role-based specializations (Admin, Head-Agent, Sub-Agent, Client), along with the relationships that govern the platform's hierarchical structure.

[Placeholder: Insert Class Diagram for Sprint 1]

Figure 4.2: Class Diagram of Sprint 1

4.1.4 Textual Description of Use Cases

4.1.4.1 User Authentication

Use Case Name	Register account
Actor(s)	Visitor
Pre-conditions	The user is not already registered. The registration page is accessible.
Post-conditions	A new Client account is created and persisted in the database with a hashed password.
Nominal Flow	
1. The Visitor navigates to the registration page (/register). 2. The Visitor fills in their full name, email address, and password. 3. The Visitor submits the form. 4. The FastAPI backend validates the input and checks that the email does not already exist. 5. The system hashes the password using the pbkdf2_sha256 algorithm. 6. A new user record is created in the PostgreSQL database with the role “Client”. 7. A success notification is displayed and the user is redirected to the login page.	
Exceptional Flows	
E1 – Email already exists: The system detects a duplicate email address and displays an error message prompting the user to log in instead. E2 – Invalid email format: The frontend validation layer rejects the submission and highlights the field with a corrective message.	

Table 4.2: Use Case – Register account

Use Case Name	Authenticate
Actor(s)	Client, Sub-Agent, Head-Agent, Admin
Pre-conditions	The user already has a registered and active account.
Post-conditions	The user is authenticated, receives a JWT token stored in an HTTP-only cookie, and is redirected to their role-specific dashboard.
Nominal Flow	
1. The user navigates to the login page (/login). 2. The user enters their email address and password. 3. The system verifies that an account exists with the provided email. 4. The system compares the submitted password against the stored hash using pbkdf2_sha256. 5. The system checks that the account is active (not disabled by an Admin or Head-Agent). 6. Upon successful validation, the backend generates a signed JWT token. 7. The token is stored in an HTTP-only cookie and the user is redirected to their role-specific dashboard (/admin, /agency, /agent, or the public catalog for Clients).	
Exceptional Flows	
E1 – Account not found: No account is associated with the entered email. A message suggests the user register instead. E2 – Incorrect password: The system alerts the user that their credentials are invalid. E3 – Account disabled: The login is denied and the user is informed that their account has been deactivated by an administrator.	

Table 4.3: Use Case – Authenticate

Use Case Name	Log Out
Actor(s)	Client, Sub-Agent, Head-Agent, Admin
Pre-conditions	The user is currently authenticated and has an active session.
Post-conditions	The JWT cookie is cleared, the Pinia auth store is reset, and the user is redirected to the login page.
Nominal Flow	
1. The authenticated user clicks the “Log Out” button in the navigation bar. 2. The frontend calls the <code>/auth/logout</code> endpoint. 3. The backend clears the HTTP-only JWT cookie. 4. The Pinia authentication store is reset on the client side. 5. A success notification is briefly displayed. 6. The user is redirected to the <code>/login</code> page.	
Exceptional Flows	
None explicitly applicable for this flow.	

Table 4.4: Use Case – Log Out

4.1.4.2 Platform Hierarchy

Use Case Name	View all users
Actor(s)	Admin
Pre-conditions	The Admin is authenticated and has accessed the Admin dashboard (<code>/admin</code>).
Post-conditions	A complete, paginated list of all registered platform users is displayed with their role, status, and account details.
Nominal Flow	
1. The Admin navigates to the User Management section of the Admin dashboard. 2. The system sends a <code>GET /auth/users</code> request to the FastAPI backend. 3. The backend verifies the Admin JWT and queries all user records from PostgreSQL. 4. The system returns the list, which is rendered in a sortable and searchable data table. 5. The Admin can see each user’s name, email, role, assigned team (for agents), and active status.	
Exceptional Flows	
E1 – No users registered: The table displays an empty state message indicating that no accounts exist yet.	

Table 4.5: Use Case – View all users

Use Case Name	Create staff account
Actor(s)	Admin
Pre-conditions	The Admin is authenticated. The email to be registered does not already exist.
Post-conditions	A new Head-Agent account is created in the database and appears in the user management table.
Nominal Flow	
1. The Admin opens the “Create New User” form on the Admin dashboard. 2. The Admin fills in the new agent’s full name, email address, and selects the role “Head-Agent” from the dropdown. 3. The Admin submits the form. 4. The backend creates the account, hashes a temporary password, and assigns the “agency” role in the database. 5. The new Head-Agent account is immediately visible in the user table with an “Active” status.	
Exceptional Flows	
E1 – Missing required fields: The system highlights the incomplete fields and prevents submission. E2 – Duplicate email: The system returns a conflict error and prompts the Admin to use a different email address.	

Table 4.6: Use Case – Create staff account

Use Case Name	Create staff account (Sub-Agent)
Actor(s)	Admin
Pre-conditions	The Admin is authenticated. At least one Head-Agent exists to be assigned as supervisor.
Post-conditions	A new Sub-Agent account is created and formally linked to the designated Head-Agent's team.
Nominal Flow	
1. The Admin opens the "Create New User" form and selects the role "Sub-Agent". 2. The Admin fills in the full name, email, and selects a Head-Agent from the supervisor dropdown. 3. The system submits the form to <code>POST /auth/register-agent</code>. 4. The backend creates the Sub-Agent account and sets the <code>head_agent_id</code> foreign key in the database. 5. The new Sub-Agent appears in the user table and is now visible in the designated Head-Agent's team view.	
Exceptional Flows	
E1 – No Head-Agent selected: The system prevents submission and requires a supervisor assignment. E2 – Duplicate email: A conflict error is returned, prompting the Admin to verify the email.	

Table 4.7: Use Case – Create staff account (Sub-Agent)

Use Case Name	Toggle user account status
Actor(s)	Admin
Pre-conditions	The Admin is authenticated and the target user account exists.
Post-conditions	The <code>is_active</code> field of the target user is toggled. If deactivated, the user's next login attempt is rejected.
Nominal Flow	
1. The Admin locates the target user in the user management table. 2. The Admin clicks the status toggle switch next to the user's record. 3. A confirmation dialog is displayed asking the Admin to confirm the action. 4. Upon confirmation, the frontend calls <code>PATCH /auth/toggle-status/{user_id}</code>. 5. The backend flips the <code>is_active</code> boolean in the database. 6. The table row updates in real-time to reflect the new "Active" or "Inactive" status.	
Exceptional Flows	
E1 – User not found: The backend returns a 404 error and the Admin is notified that the account no longer exists.	

Table 4.8: Use Case – Toggle user account status

Use Case Name	Toggle Sub-Agent account status
Actor(s)	Head-Agent
Pre-conditions	The Head-Agent is authenticated and the target Sub-Agent is assigned to their team.
Post-conditions	The Sub-Agent's <code>is_active</code> status is toggled, restricting or restoring their access to the platform.
Nominal Flow	
1. The Head-Agent navigates to the Staff Management section of the Agency dashboard (<code>/agency</code>). 2. The Head-Agent locates the target Sub-Agent in the team list. 3. The Head-Agent clicks the active/inactive toggle. 4. The system calls the backend toggle endpoint, which first verifies the Sub-Agent belongs to the requesting Head-Agent. 5. The <code>is_active</code> boolean is updated in the database. 6. The Sub-Agent's card in the team view reflects the updated status immediately.	
Exceptional Flows	
E1 – Authorization mismatch: If the Sub-Agent is not part of the requesting Head-Agent's team, the backend returns a 403 Forbidden error.	

Table 4.9: Use Case – Toggle Sub-Agent account status

4.1.5 Detailed Sequence Diagrams

This section presents the dynamic interaction flows for the critical authentication and hierarchy management processes implemented during Sprint 1.

4.1.5.1 Sequence Diagram “Register account”

This sequence illustrates the user registration process. The Visitor submits the registration form, the FastAPI backend validates the uniqueness of the email, hashes the password with `pbkdf2_sha256`, and inserts the new Client record into PostgreSQL. The process completes with a success response to the frontend.

[Placeholder: Insert Sign Up Sequence Diagram]

Figure 4.3: Sequence Diagram “Register account”

4.1.5.2 Sequence Diagram “Authenticate”

This sequence tracks the full authentication flow. The user submits their credentials, the backend verifies the email exists, compares the password hash, checks the account’s active status, generates a signed JWT token, and sets the HTTP-only cookie before redirecting the user to their role-specific route.

[Placeholder: Insert Log In Sequence Diagram]

Figure 4.4: Sequence Diagram “Authenticate”

4.1.5.3 Sequence Diagram “Log Out”

This sequence depicts the logout process. The authenticated user triggers the action, the backend clears the JWT cookie, the Pinia store is reset client-side, and the user is redirected to the public login page.

[Placeholder: Insert Log Out Sequence Diagram]

Figure 4.5: Sequence Diagram “Log Out”

4.1.5.4 Sequence Diagram “Create Head-Agent”

This sequence shows the Admin creating a new Head-Agent. After filling the creation form, the Admin triggers a `POST` request; the backend validates the payload, checks for email uniqueness, creates the user record with the appropriate role, and returns a confirmation to the Admin dashboard.

[Placeholder: Insert Create Head-Agent Sequence Diagram]

Figure 4.6: Sequence Diagram “Create Head-Agent”

4.1.5.5 Sequence Diagram “Create Sub-Agent”

This sequence shows the Admin creating a Sub-Agent linked to a Head-Agent. The critical step is the foreign key assignment (`head_agent_id`) in the database, which structurally places the new agent within a specific team hierarchy.

[Placeholder: Insert Create Sub-Agent Sequence Diagram]

Figure 4.7: Sequence Diagram “Create Sub-Agent”

4.1.5.6 Sequence Diagram “Toggle Account Status”

This sequence illustrates how the Admin enables or disables any user account. The toggle triggers a `PATCH` request; the backend flips the `is_active` boolean, and the updated status is immediately reflected in the Admin’s user management table.

[Placeholder: Insert Toggle Account Status Sequence Diagram]

Figure 4.8: Sequence Diagram “Toggle Account Status”

4.2 Implementation

This section presents the realized visual interfaces developed during Sprint 1. The screens were built using Nuxt 3 with Tailwind CSS and connect to the FastAPI backend via JWT-secured API calls.

4.2.1 Authentication Interfaces

The login interface provides a unified entry point for all user roles. After authentication, the system evaluates the JWT claims and redirects users to their designated dashboard. The registration page is available exclusively to Visitors who wish to create a Client account.

[Placeholder: Insert Screenshot of the Login Interface]

Figure 4.9: Login Interface

[Placeholder: Insert Screenshot of the Registration Interface]

Figure 4.10: Client Registration Interface

4.2.2 Admin User Management Dashboard

The Admin dashboard provides a centralized panel for monitoring all registered users. The data table supports inline status toggling, role filtering, and one-click creation of new Head-Agent or Sub-Agent accounts. Each row displays the user's name, role, email, assigned supervisor (for Sub-Agents), and current active status.

[Placeholder: Insert Screenshot of the Admin Users Dashboard]

Figure 4.11: Admin Users Management Dashboard

[Placeholder: Insert Screenshot of the Create User Form]

Figure 4.12: Create Head-Agent / Sub-Agent Form

Conclusion

Sprint 1 established the complete foundational layer of the Elite Estate platform. The implementation of JWT-based authentication, role-based routing, and the administrative hierarchy creation workflows provides a secure and structured base upon which all subsequent features are built. The Admin can now fully control the platform's user population, while the Head-Agent gains the ability to manage their own operational team.

Chapter 5

Sprint 2 - Property Management & Catalog Discovery

Introduction

Sprint 2 introduces the core business domain of the Elite Estate platform: real estate properties. This sprint enables Head-Agents to fully manage property listings — from creation with media uploads to editing and deletion — while simultaneously delivering a rich public-facing catalog with advanced filtering, pagination, interactive maps, and profile management capabilities.

5.1 Requirement Specifications

In this section, we present the Sprint 2 backlog, the corresponding use case diagram, and the complete textual descriptions of all use cases addressed during this sprint.

5.1.1 Sprint Backlog

The following product items were selected for Sprint 2, accompanied by their priority and complexity:

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
3	Platform Hierarchy	US3.6	As a Client / Sub-Agent / Head-Agent / Admin, I want to view and update my basic profile information so that my contact details are current.	Medium	13
		US3.7	As an Admin, I want to search and filter users by role so that I can manage accounts efficiently.	High	8
4	Property Mgmt	US4.1	As a Head-Agent, I want to add new properties with detailed specs so that they are available in the catalog.	High	13
		US4.2	As a Head-Agent, I want to upload high-resolution image galleries for properties so that they look appealing.	High	13
		US4.3	As a Head-Agent, I want to edit existing property details and prices so that the information stays accurate.	High	13
		US4.4	As a Head-Agent, I want to delete outdated property listings so that the catalog remains relevant.	Medium	8

Continued on next page

Table 5.1 – Continued from previous page

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
5		US4.5	As a Head-Agent, I want to filter and search through my own property listings so that I can manage them efficiently.	Medium	8
		US4.6	As a Sub-Agent, I want to consult, search, and filter my assigned properties so that I can manage them effectively.	High	8
	Catalog Discovery	US5.1	As a Visitor / Client, I want to browse the full list of available properties with pagination so that I can discover homes.	High	8
		US5.2	As a Visitor / Client, I want to filter properties by price, area, and amenities so that I can find exactly what I need.	High	13
		US5.3	As a Visitor / Client, I want to view comprehensive property details and images so that I can make an informed decision.	High	8
		US5.4	As a Visitor / Client, I want to view the geographical location of a property via interactive Leaflet maps so that I know the neighborhood.	High	13

Table 5.1: Sprint 2 Backlog

5.1.2 Use Case Diagram

The use case diagram below illustrates the full set of interactions developed during Sprint 2, covering property CRUD operations for Head-Agents, catalog browsing and filtering for Visitors and Clients, and profile management for all authenticated users.



Figure 5.1: Use Case Diagram of Sprint 2

5.1.3 Class Diagram

The class diagram below models the entities introduced in Sprint 2. The Property entity is the central addition, with relationships to the User entity (owner/assignee), the PropertyImage entity (CDN URL store), and the Amenity enumeration used for filtering.

[Placeholder: Insert Class Diagram for Sprint 2]

Figure 5.2: Class Diagram of Sprint 2

5.1.4 Textual Description of Use Cases

5.1.4.1 Profile Management

Use Case Name	Update profile information
Actor(s)	Client, Sub-Agent, Head-Agent, Admin
Pre-conditions	The user is authenticated and has navigated to the profile settings page (/profile).
Post-conditions	The user's profile information is updated in the database and the interface reflects the new values.
Nominal Flow	
1. The user accesses the profile page from the navigation menu. 2. The system retrieves and pre-fills the form with the current profile data. 3. The user modifies one or more fields (full name, phone number, profile photo). 4. The user submits the form by clicking "Save Changes". 5. The frontend sends a PUT /auth/profile request with the updated payload. 6. The backend validates the data and updates the user record in PostgreSQL. 7. A success notification is displayed and the navigation bar reflects the new profile name.	
Exceptional Flows	
E1 – Invalid field format: The system highlights the invalid field and prevents the form from being submitted until corrected. E2 – Unsupported image type: If the user uploads a non-compliant file (e.g., a PDF), the system informs them that only image formats are accepted.	

Table 5.2: Use Case – Update profile information

Use Case Name	Search/filer users
Actor(s)	Admin
Pre-conditions	The Admin is authenticated and on the User Management dashboard.
Post-conditions	The user table is filtered and displays only the records matching the selected role and search keyword.
Nominal Flow	
1. The Admin types a keyword (name or email) in the search bar of the user management table. 2. The Admin optionally selects a role filter (e.g., “Head-Agent”, “Sub-Agent”, “Client”). 3. The system dynamically filters the table client-side or sends a parameterized GET /auth/users request. 4. The table updates to display only the matching records. 5. The Admin can then act on the filtered results (e.g., toggle status, view details).	
Exceptional Flows	
E1 – No results found: The table displays an empty state message indicating no users match the search criteria.	

Table 5.3: Use Case – Search/filer users

5.1.4.2 Property Management

Use Case Name	Add new property
Actor(s)	Head-Agent
Pre-conditions	The Head-Agent is authenticated. The ImageKit CDN is operational and accessible.
Post-conditions	A new property record is created in PostgreSQL. All associated images are stored on ImageKit with their URLs persisted in the <code>property_images</code> table.
Nominal Flow	
1. The Head-Agent navigates to the “Add Property” form on the Agency dashboard (/agency). 2. The Head-Agent fills in all required fields: title, description, price, area, type (apartment/villa/land), listing type (sale/rent), city, and GPS coordinates. 3. The Head-Agent selects applicable amenities (pool, garage, balcony, etc.) from the multi-select list. 4. The Head-Agent uploads one or more property images from their local device. 5. The Head-Agent submits the form. 6. The backend calls the ImageKit SDK to upload each image and receives a CDN URL in return. 7. The backend inserts the property record and stores all CDN URLs in the <code>property_images</code> table. 8. The new listing appears immediately in the property management table.	
Exceptional Flows	
E1 – ImageKit upload failure: The CDN connection fails. The backend aborts the database transaction and notifies the Head-Agent to retry. E2 – Missing required fields: The system prevents submission and highlights all unfilled mandatory fields.	

Table 5.4: Use Case – Add new property

Use Case Name	Edit property details
Actor(s)	Head-Agent
Pre-conditions	The Head-Agent is authenticated and the property to be edited exists and belongs to them.
Post-conditions	The property record in PostgreSQL is updated with the new values, and the catalog reflects the changes immediately.
Nominal Flow	
1. The Head-Agent locates the property in their management table and clicks the “Edit” action. 2. The system opens a pre-filled edit form containing all existing property data. 3. The Head-Agent modifies the desired fields (price, description, amenities, etc.) and optionally replaces or appends images. 4. The Head-Agent submits the form. 5. The backend sends a <code>PUT /properties/{id}</code> request, validates ownership, updates the database, and handles any new image uploads via ImageKit. 6. A success toast notification is displayed.	
Exceptional Flows	
E1 – Unauthorized access: If the property does not belong to the requesting Head-Agent, the backend returns a 403 Forbidden response.	

Table 5.5: Use Case – Edit property details

Use Case Name	Delete property listing
Actor(s)	Head-Agent
Pre-conditions	The Head-Agent is authenticated and the target property belongs to them and has no pending finalization requests.
Post-conditions	The property record is removed from the database and the listing disappears from both the management view and the public catalog.
Nominal Flow	
1. The Head-Agent selects the property in the management table and clicks the “Delete” action. 2. A confirmation dialog appears, warning that this action is irreversible. 3. The Head-Agent confirms the deletion. 4. The frontend calls <code>DELETE /properties/{id}</code>. 5. The backend verifies ownership and deletes the record along with all associated images. 6. The property disappears from the table and the public catalog.	
Exceptional Flows	
E1 – Property has scheduled visits: The system warns the Head-Agent and prevents deletion until all linked visits are resolved. E2 – Unauthorized deletion: The backend returns a 403 Forbidden error if the property does not belong to the requesting agent.	

Table 5.6: Use Case – Delete property listing

Use Case Name	Upload property images
Actor(s)	Head-Agent
Pre-conditions	The Head-Agent is in the process of adding or editing a property (this use case is included in <i>Add new property</i> and <i>Edit property details</i>). The ImageKit CDN service is reachable.
Post-conditions	All selected images are uploaded to the ImageKit CDN. The returned CDN URLs are stored in the <code>property_images</code> table and are immediately available for display in the public catalog.
Nominal Flow	
1. Within the property creation or editing form, the Head-Agent accesses the image upload section. 2. The Head-Agent selects one or more image files from their local device (JPEG, PNG, or WebP formats). 3. The frontend sends the selected images to the backend endpoint (POST <code>/properties/{id}/images</code>). 4. The backend calls the ImageKit SDK (<code>backend/services/storage.py</code>) to upload each image to the CDN. 5. ImageKit processes and optimizes the images, returning a permanent CDN URL for each. 6. The backend stores each URL as a record in the <code>property_images</code> table, linked to the property. 7. The image gallery preview within the form updates to display the uploaded images.	
Exceptional Flows	
E1 – Unsupported file format: The system rejects files that are not in an accepted image format and notifies the Head-Agent to re-select a valid file. E2 – ImageKit CDN unavailable: The upload fails due to a CDN connectivity issue. The system displays an error and the Head-Agent is asked to retry.	

Table 5.7: Use Case – Upload property images

Use Case Name	Define property location on map
Actor(s)	Head-Agent
Pre-conditions	The Head-Agent is in the process of adding or editing a property (this use case is included in <i>Add new property</i>). The interactive Leaflet map component is loaded within the property form.
Post-conditions	The property record is stored with precise GPS coordinates (latitude and longitude) in the database, enabling the interactive Leaflet map to display the exact property location on the public detail page.
Nominal Flow	
1. Within the property creation form, the Head-Agent navigates to the “Location” section. 2. An interactive Leaflet map is displayed, defaulting to a general geographic view of the country. 3. The Head-Agent clicks on the map at the precise location of the property, or manually enters latitude and longitude values in the provided input fields. 4. A marker is placed on the map to confirm the selected location visually. 5. The Head-Agent can drag the marker to fine-tune the position. 6. Upon form submission, the latitude and longitude values are sent as part of the property payload and stored in the <code>properties</code> table in PostgreSQL.	
Exceptional Flows	
E1 – No location selected: The Head-Agent submits the form without defining coordinates. The property is created without GPS data; the public detail page displays a notice that the location is unavailable, and the map is not rendered.	

Table 5.8: Use Case – Define property location on map

5.1.4.3 Catalog Discovery

Use Case Name	Browse property catalog
Actor(s)	Visitor, Client
Pre-conditions	At least one active property listing exists in the database.
Post-conditions	A paginated grid of available properties is displayed.
Nominal Flow	
1. The Visitor or Client navigates to the catalog page (/properties). 2. The frontend sends a GET /properties request with default pagination parameters (e.g., page 1, limit 12). 3. The backend returns a paginated JSON payload containing property summaries (title, price, area, cover image, listing type). 4. The system renders the property cards in a responsive grid layout. 5. The user navigates between pages using the pagination controls at the bottom.	
Exceptional Flows	
E1 – No listings available: The catalog displays an empty state message indicating that no properties are currently listed.	

Table 5.9: Use Case – Browse property catalog

Use Case Name	Filter/search properties
Actor(s)	Visitor, Client
Pre-conditions	The user is on the catalog page and properties exist in the database.
Post-conditions	The property grid is updated to display only the listings that match the specified filters.
Nominal Flow	
1. The user interacts with the filter sidebar on the catalog page. 2. The user sets one or more filters: price range (min/max), surface area range, listing type (Sale/Rent), property type (Apartment/Villa/Land), and amenities (pool, garage, balcony). 3. The system sends a parameterized GET /properties?min_price=&max_price=&amenities=... request. 4. The backend constructs a dynamic SQL query with the provided filters and returns the matching properties. 5. The catalog grid refreshes to display only the filtered results with the active filter count displayed in the UI.	
Exceptional Flows	
E1 – No results: The filtering criteria yield no matching properties. The system displays an empty state and a “Clear Filters” button.	

Table 5.10: Use Case – Filter/search properties

Use Case Name	View property details
Actor(s)	Visitor, Client
Pre-conditions	The target property exists and is publicly listed.
Post-conditions	The full property detail page is rendered, including an image gallery, all specifications, and an interactive Leaflet map centered on the property's GPS coordinates.
Nominal Flow	
1. The user clicks on a property card from the catalog. 2. The frontend navigates to the dynamic route <code>/properties/[slug]</code>. 3. The system fetches the full property details via <code>GET /properties/{slug}</code>. 4. The page renders the complete image gallery (served from ImageKit CDN), all specifications (area, price, amenities, type), and the assigned Sub-Agent's contact details. 5. The Leaflet.js map component is initialized with the property's latitude and longitude, placing a marker on the exact location. 6. The user can interact with the map (zoom in/out, switch tile layers) to assess the neighborhood context.	
Exceptional Flows	
E1 – Property not found: The backend returns a 404 error and the user is redirected back to the catalog with an appropriate notification. E2 – Missing GPS coordinates: The Leaflet map displays a default view without a location marker, and a notice is shown indicating the location is unavailable.	

Table 5.11: Use Case – View property details

5.1.5 Detailed Sequence Diagrams

This section presents the interaction flows for the key property management and catalog discovery processes developed during Sprint 2.

5.1.5.1 Sequence Diagram “Add New Property”

This sequence illustrates the complete flow of a Head-Agent adding a new listing. After submitting the form, the backend calls the ImageKit SDK to upload each image to the CDN, receives the resulting URLs, and inserts the property record along with the image references into PostgreSQL. The listing becomes immediately accessible in the public catalog.

[Placeholder: Insert Add New Property Sequence Diagram]

Figure 5.3: Sequence Diagram “Add New Property”

5.1.5.2 Sequence Diagram “Edit Property”

This sequence shows the Head-Agent editing an existing listing. The backend first verifies ownership of the property before applying any updates to prevent unauthorized modifications. Modified images are re-uploaded to ImageKit and their new CDN URLs replace the old references.

[Placeholder: Insert Edit Property Sequence Diagram]

Figure 5.4: Sequence Diagram “Edit Property”

5.1.5.3 Sequence Diagram “Filter Catalog”

This sequence details how a Visitor applies filters on the catalog page. The frontend constructs a parameterized API query; the backend uses SQLAlchemy’s dynamic filtering to build and execute the SQL query against PostgreSQL, returning only the listings that match all specified criteria.

[Placeholder: Insert Filter Catalog Sequence Diagram]

Figure 5.5: Sequence Diagram “Filter Catalog”

5.1.5.4 Sequence Diagram “View Property Details”

This sequence shows a user opening a specific property page. The slug-based route triggers a backend fetch of the complete property object, which includes joining the images and amenities tables. The Leaflet map component is initialized with the GPS coordinates returned by the API.

[Placeholder: Insert View Property Details Sequence Diagram]

Figure 5.6: Sequence Diagram “View Property Details”

5.2 Implementation

This section presents the frontend interfaces realized during Sprint 2. All components are built in Nuxt 3 with reactive Vue composition API patterns and connect to the FastAPI backend in real time.

5.2.1 Agency Property Management Dashboard

The Agency dashboard allows Head-Agents to manage their entire portfolio. The interface features a searchable and filterable property table with inline action buttons for editing, dele-

tion, and status changes. The “Add Property” form is accessible via a prominent button and includes a multi-step layout for specifications, image uploads, and amenity selection.

[Placeholder: Insert Screenshot of the Agency Property Table]

Figure 5.7: Head-Agent Property Management Table

[Placeholder: Insert Screenshot of the Add Property Form]

Figure 5.8: Add New Property Form with ImageKit Upload

5.2.2 Public Property Catalog

The public catalog presents a responsive grid of property cards, each displaying the cover image (served from ImageKit CDN), the listing type badge (Sale/Rent), title, price, and surface area. The sidebar filter panel allows users to refine results in real time across six filter dimensions.

[Placeholder: Insert Screenshot of the Public Catalog Grid with Filters]

Figure 5.9: Public Property Catalog with Filters

5.2.3 Property Detail Page with Leaflet Map

The property detail page displays the full image gallery in a swipeable carousel, a structured specification panel (area, price, type, amenities), the assigned agent's contact information, and an interactive Leaflet.js map tile pinpointing the exact property location. The map supports standard zoom and pan interactions with OpenStreetMap tiles.

[Placeholder: Insert Screenshot of the Property Detail Page with Leaflet Map]

Figure 5.10: Property Detail Page with Interactive Leaflet Map

Conclusion

Sprint 2 successfully delivered the core business domain of the Elite Estate platform. Head-Agents can now create, manage, and delete property listings with professional media handling via ImageKit, while Visitors and Clients enjoy a rich, interactive catalog discovery experience with advanced filtering and geographic visualization. The platform now contains fully dynamic, manageable content ready for AI enhancement in the next sprint.

Chapter 6

Sprint 3 - AI Intelligence, TAIA & Visit Management

Introduction

Sprint 3 introduces the most technically advanced features of the Elite Estate platform. It integrates Google Gemini 1.5 with an Ollama-powered RAG architecture to deliver an intelligent conversational AI, deploys the TAIA (Telegram AI Assistant) bot for Client interactions via Telegram, implements the Telegram account pairing flow for Clients, and establishes the full visit management lifecycle within the Sub-Agent dashboard. This sprint also delivers Sub-Agent performance analytics views and transaction report download capabilities.

6.1 Requirement Specifications

In this section, we present the Sprint 3 backlog, the corresponding use case diagram, and the complete textual descriptions of all use cases addressed during this sprint.

6.1.1 Sprint Backlog

The following product items were selected for Sprint 3, accompanied by their priority and complexity:

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
6	AI Intelli- gence	US6.1	As a Visitor / Client, I want the AI to retrieve accurate property context (RAG) before answering so that I can receive truthful, contextual information.	High	21
		US6.2	As a Visitor / Client, I want the AI to perform semantic vector searches so that I can discover and get recommendations for highly similar listings.	High	21
7	TAIA	US7.1	As a Client, I want to link my Telegram account via a pairing code so that TAIA can identify me and send personalized notifications.	High	13
		US7.2	As a Visitor / Client, I want the AI to parse reschedule or cancellation requests and update the appointment record instantly so that appointment records are synchronized.	High	13
8	Visit Mgmt	US8.1	As a Sub-Agent, I want new TAIA-booked visits to automatically appear on my dashboard as “Scheduled”.	High	13

Continued on next page

Table 6.1 – *Continued from previous page*

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
		US8.2	As a Sub-Agent, I want to manually change the status of an appointment to “Finished” or “Cancelled” from my dashboard.	High	8
		US8.3	As a Sub-Agent, I want to filter my assigned visits so that I can organize my daily schedule efficiently.	Medium	8

Table 6.1: Sprint 3 Backlog

6.1.2 Use Case Diagram

The use case diagram below illustrates the interactions between the Client, the AI Agent (acting as a system actor), and the Sub-Agent. It covers the full TAIA conversational loop, the Telegram pairing flow, and the visit management lifecycle on the Sub-Agent’s operational dashboard.

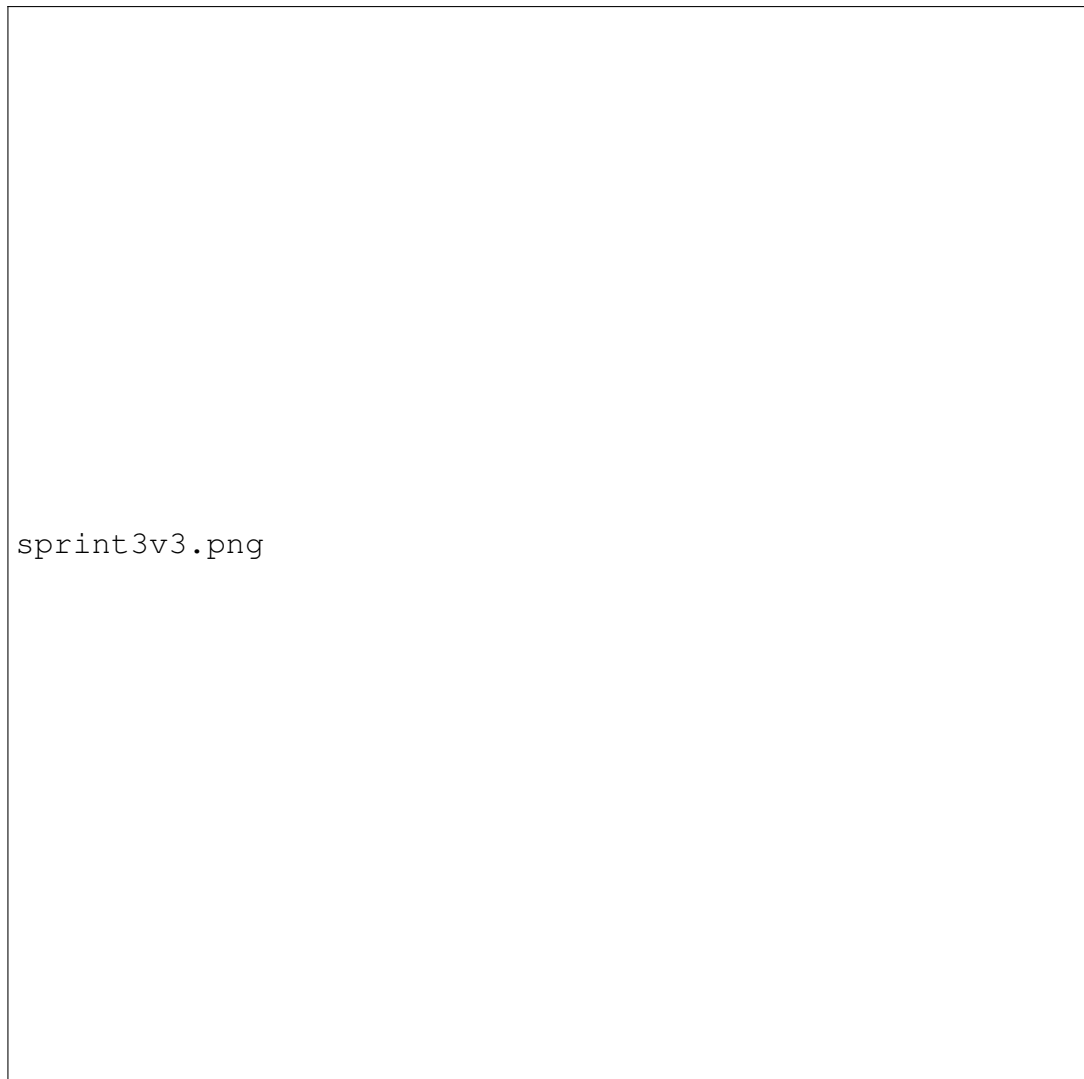


Figure 6.1: Use Case Diagram of Sprint 3

6.1.3 Class Diagram

The class diagram below models the entities introduced in Sprint 3. The Visit entity is the central addition, with relationships to the Property entity (the subject of the visit), the User entity (Client and Sub-Agent), and the vector embedding column added to the Property table for semantic search.

[Placeholder: Insert Class Diagram for Sprint 3]

Figure 6.2: Class Diagram of Sprint 3

6.1.4 Textual Description of Use Cases

6.1.4.1 AI Intelligence

Use Case Name	Ask AI assistant about property
Actor(s)	AI Agent
Pre-conditions	The property database is populated and pgvector embeddings have been generated for all listings using the Ollama <code>nomic-embed-text</code> model.
Post-conditions	The AI Agent returns a contextually grounded answer based on real property data, preventing hallucinated responses.
Nominal Flow	
1. The Client sends a natural language question to TAIA (e.g., “Tell me about properties with a pool under 300,000 TND”). 2. The n8n TAIA Smart Agent workflow receives the message via the Telegram webhook. 3. The workflow calls the FastAPI backend at <code>POST /properties/rag-search</code> with the query string. 4. The backend uses Ollama to convert the query into a 768-dimensional embedding vector. 5. PostgreSQL’s pgvector extension runs an exact L2 distance query (flat search) and returns the top-5 semantically matching properties. 6. The retrieved property context is injected into the Google Gemini 1.5 system prompt. 7. Gemini formulates an accurate, context-grounded answer referencing only the retrieved data. 8. The answer is relayed back to the Client via TAIA.	
Exceptional Flows	
E1 – No matching properties found: The vector search returns no results above the similarity threshold. The AI informs the Client that no matching listings are currently available. E2 – Ollama service unavailable: The embedding call fails. The system falls back to a keyword-based search and notifies the Admin of the service outage.	

Table 6.2: Use Case – Ask AI assistant about property

Use Case Name	Search property
Actor(s)	AI Agent, Visitor, Client
Pre-conditions	The pgvector extension is active and property embeddings are up to date in the database.
Post-conditions	A ranked list of semantically similar properties is returned to the requesting actor, ordered by ascending L2 distance from the query vector.
Nominal Flow	
1. A Visitor types a conceptual search query in the web catalog search bar (e.g., “quiet countryside villa”). 2. The frontend sends the query to <code>POST /properties/semantic-search</code>. 3. The backend calls the local Ollama API to generate a query vector. 4. The backend executes the <code>pgvector l2_distance</code> function against the <code>embedding</code> column of the <code>properties</code> table. 5. The database returns the five closest property records sorted by semantic similarity. 6. The frontend renders the results as a ranked list of property cards, each labeled with its conceptual relevance.	
Exceptional Flows	
E1 – Empty query string: The system displays a validation message requesting the user to enter a search term. E2 – Ollama timeout: If the embedding model does not respond within the timeout window, the user is shown an error and advised to try again.	

Table 6.3: Use Case – Search property

6.1.4.2 TAIA – Telegram AI Assistant

Use Case Name	Integrate telegram
Actor(s)	Client, AI Agent
Pre-conditions	The Client's Telegram account is linked to the platform via a pairing code. The n8n TAIA Smart Agent workflow is running and the webhook is active.
Post-conditions	The Client receives a contextually accurate, personalized response from the AI Agent via Telegram.
Nominal Flow	
1. The Client sends a message to the Elite Estate Telegram bot. 2. The Telegram API forwards the message to the n8n webhook endpoint via HTTPS (tunnelled via ngrok). 3. The n8n workflow identifies the Client by their <code>telegram_chat_id</code> via a FastAPI lookup. 4. The workflow appends the message to the conversation history and builds the RAG context by calling the semantic search endpoint. 5. The workflow sends the enriched prompt (user message + property context + chat history) to Google Gemini 1.5 Flash. 6. Gemini generates a structured response. 7. n8n sends the response back to the Client's Telegram chat via the Telegram Bot API.	
Exceptional Flows	
E1 – Client not paired: The system cannot identify the sender's Telegram ID. TAIA responds with a message directing the Client to pair their account via the web platform. E2 – n8n workflow error: The workflow encounters an unhandled exception. TAIA responds with a friendly error message and the incident is logged.	

Table 6.4: Use Case – Integrate telegram

Use Case Name	Pair telegram
Actor(s)	Client
Pre-conditions	The Client is authenticated on the web platform. The Client has a Telegram account.
Post-conditions	The Client's <code>telegram_chat_id</code> is stored in the database, enabling TAIA to identify them in future conversations.
Nominal Flow	
1. The Client navigates to their profile page (<code>/profile</code>) and clicks the “Link Telegram Account” button. 2. The system calls <code>GET /auth/generate-code</code> and generates a unique, time-limited pairing code (e.g., <code>ELITE-7X4Q</code>). 3. The pairing code is displayed to the Client on the profile page. 4. The Client opens the Elite Estate Telegram bot and sends the command <code>/link ELITE-7X4Q</code>. 5. The n8n workflow receives the message, extracts the code, and calls <code>POST /auth/pair-telegram</code> with the code and the sender's <code>chat_id</code>. 6. The backend matches the code to the Client's account and stores the <code>telegram_chat_id</code> in the database. 7. TAIA responds to the Client in Telegram confirming the successful pairing. 8. The profile page updates to show a “Telegram Linked” badge.	
Exceptional Flows	
E1 – Expired pairing code: The Client sends a code that has passed the expiry window. TAIA informs the Client to generate a new code from the profile page. E2 – Code already used: The code has already been consumed. TAIA notifies the Client and the platform prompts them to generate a fresh code.	

Table 6.5: Use Case – Pair telegram

Use Case Name	Schedule visit via AI assistant
Actor(s)	Client, AI Agent
Pre-conditions	The Client's Telegram account is paired. The target property exists and an active Sub-Agent is assigned.
Post-conditions	A new visit record with status "Scheduled" is created in the database, and the visit appears on the Sub-Agent's dashboard calendar.
Nominal Flow	
1. The Client messages TAIA expressing intent to visit a property (e.g., "I want to visit the Gammarth villa tomorrow at 3 PM"). 2. The AI Agent extracts the temporal intent (date and time) and the property reference from the message. 3. The n8n workflow queries the backend API to verify availability of the requested slot. 4. The slot is confirmed as free. 5. The workflow calls <code>POST /visits</code> to create the visit record in PostgreSQL with status "Scheduled". 6. The visit seamlessly appears in the Sub-Agent's dashboard calendar. 7. TAIA confirms the booking details to the Client in Telegram, including the date, time, and property address.	
Exceptional Flows	
E1 – Slot unavailable: The Sub-Agent's calendar shows a conflict. TAIA informs the Client and suggests two alternative available time slots. E2 – Ambiguous property reference: The AI cannot resolve the property from the message. TAIA asks the Client to clarify the property name or provide the listing ID.	

Table 6.6: Use Case – Schedule visit via AI assistant

Use Case Name	Reschedule visit via AI assistant
Actor(s)	Client, AI Agent
Pre-conditions	An existing visit with status “Scheduled” is linked to the Client’s account. The Client is interacting with TAIA.
Post-conditions	The visit record is updated in PostgreSQL (new date/time for reschedule, or status “Cancelled”) and the dashboard calendar is updated or cleared accordingly.
Nominal Flow	
1. The Client sends a reschedule or cancellation intent to TAIA (e.g., “Can we move my visit to Friday at 10 AM?” or “Please cancel my visit”). 2. The AI Agent identifies the action type (reschedule or cancel) and the target visit from the context. 3. For reschedule: The workflow checks the Sub-Agent’s calendar for the new slot. If free, it calls <code>PATCH /visits/{id}</code> with the updated datetime. 4. For cancellation: The workflow calls <code>PATCH /visits/{id}</code> setting the status to “Cancelled”. 5. The calendar on the dashboard automatically reflects the updated or deleted visit. 6. TAIA confirms the change to the Client.	
Exceptional Flows	
E1 – New slot conflict (reschedule): The requested new time is already booked. TAIA proposes available alternatives. E2 – Visit already completed or cancelled: The AI cannot modify a finalized visit and informs the Client accordingly.	

Table 6.7: Use Case – Reschedule visit via AI assistant

Use Case Name	Book visit
Actor(s)	Client, AI Agent
Pre-conditions	The Client is paired with their Telegram account. The target property exists, has an assigned active Sub-Agent, and the requested time slot is available in the Sub-Agent's schedule. A visit scheduling intent has been confirmed between the Client and TAIA.
Post-conditions	A new visit record with status "Scheduled" is created in the PostgreSQL database. A corresponding event appears in the Sub-Agent's dashboard calendar. The Client receives a booking confirmation from TAIA via Telegram.
Nominal Flow	
1. Following a conversation with TAIA, the Client confirms their intent to book a property visit at a specific date and time. 2. The n8n TAIA Smart Agent workflow receives the finalized booking intent from the Client. 3. The workflow calls <code>POST /visits</code> to create a new visit record in PostgreSQL with the status "Scheduled", linking the Client, the assigned Sub-Agent, and the property. 4. Simultaneously, the visit information is fully logged so it appears on the Sub-Agent's dashboard calendar. 5. TAIA sends a booking confirmation to the Client in Telegram, summarizing the property name, address, visit date, and time.	
Exceptional Flows	
E1 – Booking fails: The backend API returns an error. TAIA informs the Client that the booking could not be finalized and suggests retrying. E2 – Sub-Agent no longer available: If the Sub-Agent's account is deactivated between the conversation and booking, the system assigns the visit to the next available Sub-Agent in the team.	

Table 6.8: Use Case – Book visit

Use Case Name	Abandon visit via AI assistant
Actor(s)	Client, AI Agent
Pre-conditions	The Client is paired with Telegram. An active visit with status “Scheduled” is linked to the Client’s account. The Client communicates an abandonment intent to TAIA.
Post-conditions	The visit record’s status is updated to “Cancelled” in the database. The corresponding event is removed from the dashboard calendar. The Client receives a cancellation acknowledgment from TAIA.
Nominal Flow	
1. The Client messages TAIA expressing their wish to abandon (cancel) a scheduled visit (e.g., “I want to cancel my visit to the Tunis apartment”). 2. The AI Agent identifies the relevant visit from the Client’s booking history. 3. TAIA asks the Client to confirm the cancellation of the identified visit. 4. The Client confirms. 5. The n8n workflow calls <code>PATCH /visits/{id}</code> setting the status to “Cancelled”. 6. The workflow updates the visit status, clearing it from the Sub-Agent’s active schedule. 7. TAIA confirms the cancellation to the Client in Telegram.	
Exceptional Flows	
E1 – No active visit found: The AI cannot identify a scheduled visit to cancel. TAIA informs the Client and asks them to clarify. E2 – Visit already completed: The visit is in a terminal state. TAIA informs the Client that the visit can no longer be cancelled.	

Table 6.9: Use Case – Abandon visit via AI assistant

6.1.4.3 Visit Management

Use Case Name	View AI-booked visits
Actor(s)	Sub-Agent
Pre-conditions	The Sub-Agent is authenticated. At least one visit has been booked for a property assigned to them.
Post-conditions	All visits linked to the Sub-Agent are displayed in a structured table with status badges.
Nominal Flow	
1. The Sub-Agent logs in and navigates to the “My Visits” section of the Agent dashboard (/agent). 2. The system sends GET /visits/my-visits to the backend. 3. The backend queries all visits where agent_id matches the authenticated Sub-Agent, joining property and client data. 4. The results are rendered in a paginated table showing: client name, property title, scheduled date/time, and status (Scheduled/Finished/Cancelled). 5. The Sub-Agent can sort and filter the table by status or date range.	
Exceptional Flows	
E1 – No visits assigned: The table displays an empty state informing the Sub-Agent that no visits have been scheduled yet.	

Table 6.10: Use Case – View AI-booked visits

Use Case Name	Update visit status
Actor(s)	Sub-Agent
Pre-conditions	A visit with status “Scheduled” exists and is assigned to the authenticated Sub-Agent.
Post-conditions	The visit status is updated to “Finished” or “Cancelled” in the database, and the Sub-Agent’s analytics statistics are recalculated.
Nominal Flow	
1. The Sub-Agent locates the visit in the visits table on the Agent dashboard. 2. The Sub-Agent clicks the status action button (“Mark as Finished” or “Cancel Visit”). 3. A confirmation dialog is shown. 4. Upon confirmation, the frontend calls <code>PATCH /visits/{id}/status</code> with the new status value. 5. The backend updates the <code>status</code> field in the visits table. 6. If marked “Finished”, the Sub-Agent’s analytics counter for completed visits is incremented. 7. The table row’s status badge updates in real time.	
Exceptional Flows	
E1 – Visit already finalized: The system prevents re-updating a visit that is already in a terminal state (“Finished” or “Cancelled”) and displays an appropriate message. E2 – Unauthorized action: The backend returns a 403 error if the Sub-Agent attempts to update a visit not assigned to them.	

Table 6.11: Use Case – Update visit status

Use Case Name	View personal analytics
Actor(s)	Sub-Agent
Pre-conditions	The Sub-Agent is authenticated and has at least one finalized visit.
Post-conditions	A personal analytics dashboard is displayed showing counts of total visits, finished visits, and cancellation rate.
Nominal Flow	
1. The Sub-Agent accesses the statistics section of their dashboard. 2. The system calls <code>GET /statistics/agent</code> with the authenticated Sub-Agent's token. 3. The backend aggregates visit records for the agent: total count, finished count, and cancelled count. 4. The analytics widgets are rendered on the dashboard: Total Visits, Finished Visits, Cancelled Visits, and a Success Rate percentage. 5. The statistics are computed in real time from the current database state.	
Exceptional Flows	
E1 – No completed visits: All analytics counters display zero, and a message encourages the Sub-Agent to complete their first visit.	

Table 6.12: Use Case – View personal analytics

Use Case Name	Consult visits calendar
Actor(s)	Sub-Agent
Pre-conditions	The Sub-Agent is authenticated. At least one visit is recorded in the database for the Sub-Agent's assigned properties.
Post-conditions	The Sub-Agent's personal visit schedule is displayed in an interactive calendar view, allowing them to plan and organize their working schedule efficiently.
Nominal Flow	
1. The Sub-Agent navigates to the "Calendar" section of the Agent dashboard (/agent). 2. The system calls <code>GET /visits/my-visits</code> with the Sub-Agent's authentication token. 3. The backend returns all visit records assigned to the Sub-Agent, including status, date, time, property name, and client details. 4. The frontend renders the visits as events on an interactive calendar component (monthly/weekly view), color-coded by status: Scheduled (blue), Finished (green), Cancelled (red). 5. The Sub-Agent can click on any calendar event to view the full visit details. 6. The Sub-Agent can navigate between weeks or months using the calendar navigation controls.	
Exceptional Flows	
E1 – No visits in calendar: The calendar is displayed empty for the selected period, with a message indicating no appointments are scheduled yet.	

Table 6.13: Use Case – Consult visits calendar

Use Case Name	Download transaction reports
Actor(s)	Admin, Head-Agent
Pre-conditions	The requesting user is authenticated with the Admin or Head-Agent role. Finalized transactions exist in the database.
Post-conditions	A PDF transaction report is generated server-side and downloaded to the user's device.
Nominal Flow	
1. The Admin or Head-Agent accesses the Reports section of their dashboard. 2. The user clicks the "Download Report" button. 3. The frontend calls <code>GET /reports/transactions</code> with a Bearer token. 4. The FastAPI backend queries all finalized (Sold/Rented) properties with their associated agent and client data. 5. The backend compiles the data into a formatted PDF using a server-side library. 6. The PDF file is streamed as a binary response and downloaded by the user's browser.	
Exceptional Flows	
E1 – No transactions available: The backend returns an empty report with a message indicating no finalized transactions exist. E2 – Insufficient permissions: A Sub-Agent or unauthenticated user attempting to access the endpoint receives a 403 Forbidden response.	

Table 6.14: Use Case – Download transaction reports

6.1.5 Detailed Sequence Diagrams

This section presents the dynamic interaction flows for the AI intelligence pipeline and the visit management lifecycle implemented during Sprint 3.

6.1.5.1 Sequence Diagram "RAG Semantic Search Chat"

This sequence illustrates the complete Retrieval-Augmented Generation pipeline. When the Client messages TAIA, the n8n workflow converts the query to a vector via Ollama, runs the pgvector L2 distance query, injects the retrieved property context into the Gemini system prompt, and returns a grounded response. This entire loop is designed to complete within the n8n workflow's execution time limit.

[Placeholder: Insert RAG Semantic Search Sequence Diagram]

Figure 6.3: Sequence Diagram “RAG Semantic Chat with TAIA”

6.1.5.2 Sequence Diagram “Link Telegram via Pairing Code”

This sequence details the Telegram account pairing mechanism. The Client generates a time-limited code from the web platform, sends it to the Telegram bot, and the n8n workflow pairs the `telegram_chat_id` to the Client’s database record, completing the integration.

[Placeholder: Insert Telegram Pairing Sequence Diagram]

Figure 6.4: Sequence Diagram “Telegram Account Pairing”

6.1.5.3 Sequence Diagram “Book Visit via TAIA”

This sequence shows the AI autonomously creating a visit appointment. TAIA extracts the user’s intent, queries the backend API for availability, creates the visit record in PostgreSQL, and confirms the booking to the Client – all within a single n8n workflow execution.

[Placeholder: Insert Book Visit via TAIA Sequence Diagram]

Figure 6.5: Sequence Diagram “Book Visit via TAIA”

6.1.5.4 Sequence Diagram “Update Visit Status”

This sequence shows the Sub-Agent manually updating a visit’s status from their dashboard. The backend validates ownership before applying the status change, and if the visit is marked “Finished”, the statistics service recalculates the Sub-Agent’s analytics metrics.

[Placeholder: Insert Update Visit Status Sequence Diagram]

Figure 6.6: Sequence Diagram “Update Visit Status”

6.2 Implementation

This section presents the realized interfaces for the AI-powered features and the Sub-Agent visit management dashboard developed during Sprint 3.

6.2.1 TAIA Web Interface and Pairing Flow

The homepage (`/index.vue`) features the TAIA web interface – a real-time chat widget where Visitors can interact with the AI Assistant directly from the catalog page. The conversation uses the same RAG pipeline as the Telegram bot. The profile page presents the

Telegram pairing section where Clients can generate their unique pairing code and monitor the linked status.

[Placeholder: Insert Screenshot of the TAIA Web Chat Interface]

Figure 6.7: TAIA Web Chat Interface with Semantic Search

[Placeholder: Insert Screenshot of the Telegram Pairing Section on Profile Page]

Figure 6.8: Telegram Account Pairing Section

6.2.2 Sub-Agent Visits Dashboard

The Agent dashboard (`/agent`) displays a structured visit management table, showing all appointments assigned to the logged-in Sub-Agent. Each row includes client name, property, date, and a status badge. Inline action buttons allow the agent to update the visit's status. analytics summary cards at the top of the page display the agent's total, finished, and cancelled visit counts.

[Placeholder: Insert Screenshot of the Sub-Agent Visits Table]

Figure 6.9: Sub-Agent Visit Management Dashboard

[Placeholder: Insert Screenshot of the Sub-Agent Analytics Statistics Cards]

Figure 6.10: Sub-Agent Personal Analytics Statistics

Conclusion

Sprint 3 successfully bridged cutting-edge AI with real estate operations. By combining Retrieval-Augmented Generation, local vector embeddings via Ollama, and the Telegram messaging channel, the platform now delivers an intelligent, always-available conversational assistant (TAIA). The full visit lifecycle — from AI-driven booking through calendar integration to Sub-Agent status management — is now fully operational, eliminating traditional scheduling overhead entirely.

Chapter 7

Sprint 4 - Notifications, Analytics & Finalization

Introduction

Sprint 4 concludes the Elite Estate development cycle by delivering the platform's oversight and orchestration layer. This sprint implements the fully automated notification system (meeting reminders via TAIA and Email, and transaction confirmation emails), the comprehensive team and personal calendar views for Head-Agents, the analytics dashboards for both Head-Agents and the Admin, and the complete transaction finalization workflow where Sub-Agents submit sale or rental requests, Head-Agents formally approve or reject them, and Sub-Agents complete the approved transactions to finalize deals.

7.1 Requirement Specifications

In this section, we present the Sprint 4 backlog, the corresponding use case diagram, and the complete textual descriptions of all use cases addressed during this sprint.

7.1.1 Sprint Backlog

The following product items were selected for Sprint 4, accompanied by their priority and complexity:

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
9	Notifications	US9.1	As a Client / Sub-Agent, I want to receive automated reminder notifications via TAIA and Email prior to our scheduled meeting.	High	21
		US9.2	As a Client / Sub-Agent / Head-Agent / Admin, I want to receive automated emails for my specific transaction operations (e.g., Client/Admin receives reports, Sub/Head-Agent receives responses).	High	13
10	Analytics & Oversight	US10.1	As a Head-Agent, I want to view a team calendar in my dashboard displaying all appointments of my assigned Sub-Agents with statuses and filters.	High	13
		US10.2	As a Head-Agent, I want to consult my personal appointment calendar in the dashboard so that I can track my own schedule.	High	13
		US10.3	As a Head-Agent, I want to view my analytics statistics and team performance so that I can evaluate my operations.	High	13

Continued on next page

Table 7.1 – Continued from previous page

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
		US10.4	As an Admin, I want to view a analytics statistics dashboard of platform growth and total transactions to measure global success.	High	13
		US10.5	As a Sub-Agent, I want to view my personal analytics statistics based on completed visits so that I can track my performance.	Medium	8
		US10.6	As an Admin or Head-Agent, I want to download detailed transaction reports so that I can keep external records of platform deals.	High	13
11	Finalization	US11.1	As a Sub-Agent, I want to submit transaction requests and complete them once approved (Request Sale/Rent and Complete Transaction).	High	13
		US11.2	As a Head-Agent, I want to approve a transaction request, allowing the Sub-Agent to proceed with completion.	High	13

Continued on next page

Table 7.1 – *Continued from previous page*

ID	Feature	PI/US ID	Product Item / User Story	Priority	Comp.
		US11.3	As a Head-Agent, I want to refuse a transaction request, which will notify the Sub-Agent via Email of the refusal.	Medium	8
		US11.4	As a Head-Agent, I want to consult my rented and sold properties so that I can track my historically finalized deals easily.	Medium	8

Table 7.1: Sprint 4 Backlog

7.1.2 Use Case Diagram

The use case diagram below illustrates the finalization workflow between Sub-Agents and Head-Agents, the automated notification triggers, and the analytics consultation capabilities of Head-Agents and Admins through their respective dashboards.

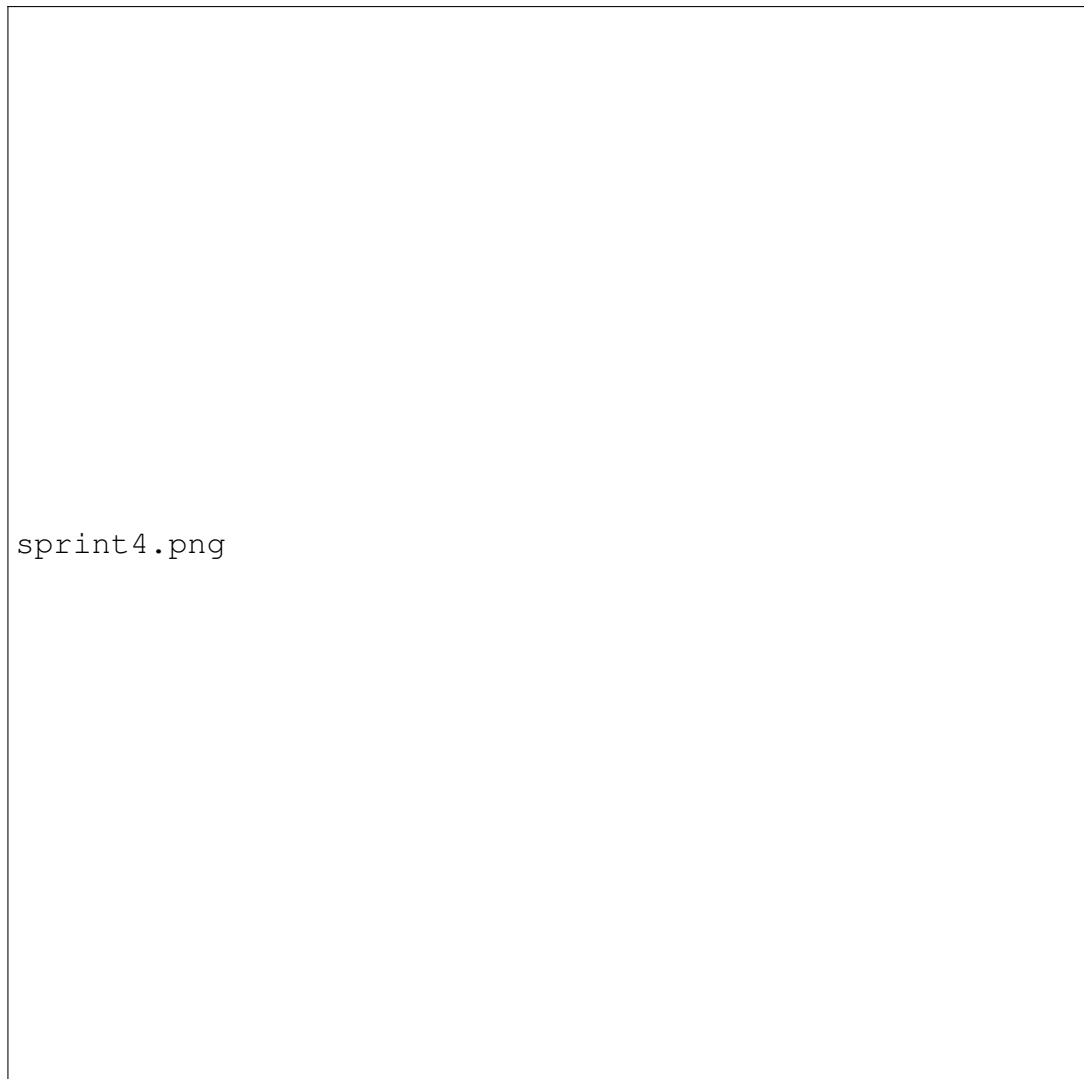


Figure 7.1: Use Case Diagram of Sprint 4

7.1.3 Class Diagram

The class diagram below models the new entities introduced in Sprint 4: the TransactionRequest entity (linking Sub-Agent, Head-Agent, and Property), and the structural relationships supporting the calendar and analytics aggregations.

[Placeholder: Insert Class Diagram for Sprint 4]

Figure 7.2: Class Diagram of Sprint 4

7.1.4 Textual Description of Use Cases

7.1.4.1 Notification System

Use Case Name	Receive visit reminders via Telegram
Actor(s)	Client, Sub-Agent, AI Agent
Pre-conditions	A visit with status “Scheduled” exists with a datetime within the next 60 minutes. Both the Client’s Telegram account and the Sub-Agent’s email address are on file.
Post-conditions	The Client receives a TAIA reminder message via Telegram. The Sub-Agent receives a reminder via email.
Nominal Flow	
1. The n8n Meeting Reminder Service workflow triggers on its configured CRON schedule (e.g., every 30 minutes). 2. The workflow calls <code>GET /visits/upcoming</code> to retrieve all visits scheduled within the next hour. 3. For each visit found, the workflow dispatches two notifications in parallel: a. A TAIA Telegram message to the Client’s <code>telegram_chat_id</code>, including the property name, address, and visit time. b. An email via the SMTP email service (<code>backend/services/email.py</code>) to the Sub-Agent, containing the client name and visit details. 4. Each notification is logged and the workflow execution completes.	
Exceptional Flows	
E1 – No upcoming visits: The workflow finds no qualifying records and terminates silently without sending any notifications. E2 – Telegram delivery failure: The Telegram API call fails. The failure is logged in the n8n workflow execution history and the SMTP fallback email is still dispatched.	

Table 7.2: Use Case – Receive visit reminders via Telegram

Use Case Name	Receive approval email for transaction request
Actor(s)	Sub-Agent, Head-Agent, Admin, Client
Pre-conditions	A transaction has been completed by the Sub-Agent (following Head-Agent approval), or a transaction request has been rejected by the Head-Agent.
Post-conditions	All relevant parties receive a role-specific email notification confirming the outcome or completion of the transaction.
Nominal Flow	
1. The Sub-Agent completes an approved transaction, or a Head-Agent rejects a finalization request on the Agency dashboard. 2. The backend calls the email service (<code>backend/services/email.py</code>) asynchronously. 3. If completed: a. The Admin receives an email with a transaction summary report in attachment. b. The Client receives an email confirming the property sale or rental. 4. If rejected: a. The Sub-Agent receives an email with the rejection notification. 5. All emails are sent via SMTP using the configured email account.	
Exceptional Flows	
E1 – SMTP service unavailable: The email delivery fails. The failure is logged server-side and the transaction outcome is still persisted in the database. A retry mechanism attempts re-delivery after a configurable delay.	

Table 7.3: Use Case – Receive approval email for transaction request

7.1.4.2 Analytics & Oversight

Use Case Name	Consult team calendar
Actor(s)	Head-Agent
Pre-conditions	The Head-Agent is authenticated and has at least one Sub-Agent assigned to their team.
Post-conditions	An interactive calendar view is displayed, populated with all visit appointments from the Head-Agent's Sub-Agents, filterable by status and date range.
Nominal Flow	
1. The Head-Agent navigates to the "Team Calendar" section of the Agency dashboard. 2. The system calls <code>GET /visits/team-calendar</code> with the Head-Agent's token. 3. The backend queries all visits linked to Sub-Agents who are members of the requesting Head-Agent's team. 4. The visit data is returned as a list of calendar events with date, time, status, agent name, and property name. 5. The frontend renders the events on an interactive calendar component (monthly/weekly view). 6. The Head-Agent applies status filters (e.g., show only "Scheduled" visits) or navigates between months. 7. The calendar dynamically updates to reflect the selected filters.	
Exceptional Flows	
E1 – No Sub-Agents assigned: The calendar renders in an empty state with a message recommending the Head-Agent contact the Administrator for team assignment. E2 – No visits scheduled: The calendar is displayed but empty, with a message indicating no appointments exist for the selected period.	

Table 7.4: Use Case – Consult team calendar

Use Case Name	Consult visits calendar
Actor(s)	Head-Agent
Pre-conditions	The Head-Agent is authenticated on the Agency dashboard.
Post-conditions	The Head-Agent's personal appointment schedule (own visits and supervisory events) is rendered on an individual calendar view.
Nominal Flow	
1. The Head-Agent selects the "My Calendar" tab within the Agency dashboard. 2. The system calls <code>GET /visits/my-calendar</code> using the Head-Agent's authenticated token. 3. The backend returns only the visits and events directly associated with the Head-Agent. 4. The calendar component renders the personal schedule, with each event color-coded by status. 5. The Head-Agent can click on any event to view its full details (client, property, time).	
Exceptional Flows	
E1 – No personal events: The personal calendar is displayed empty with a message indicating no appointments are scheduled for the selected period.	

Table 7.5: Use Case – Consult visits calendar

Use Case Name	consult analytics
Actor(s)	Head-Agent
Pre-conditions	The Head-Agent is authenticated and has an active team with recorded activities.
Post-conditions	A analytics summary panel is displayed showing total listings, team visits, finalized transactions, and individual Sub-Agent performance comparisons.
Nominal Flow	
1. The Head-Agent accesses the “Statistics” section of the Agency dashboard. 2. The frontend calls GET /statistics/agency with the Head-Agent’s token. 3. The backend aggregates data: total properties owned, total visits by all Sub-Agents, finalized transactions (Sold + Rented), and a per-agent visit breakdown. 4. analytics cards and chart widgets are rendered: total listings count, total team visits, success rate, and a bar chart comparing Sub-Agent performance. 5. The Head-Agent can identify their top-performing Sub-Agent and monitor overall team efficiency.	
Exceptional Flows	
E1 – No data available: All analytics counters display zero and a message indicates there is no activity recorded yet for the team.	

Table 7.6: Use Case – consult analytics

Use Case Name	consult analytics
Actor(s)	Admin
Pre-conditions	The Admin is authenticated on the Admin dashboard.
Post-conditions	A global statistics panel is displayed showing platform-wide user counts, total listings, total visits, and finalized transaction volume.
Nominal Flow	
1. The Admin accesses the “Statistics” section of the Admin dashboard. 2. The frontend calls <code>GET /statistics/admin</code> with the Admin’s token. 3. The backend computes platform-wide aggregates: total registered users (by role), total property listings, total scheduled and completed visits, and total finalized transactions. 4. The Admin’s dashboard renders four primary analytics cards (Total Users, Total Listings, Total Visits, Total Transactions) alongside a trend chart visualizing growth over time. 5. The Admin gains a real-time, comprehensive view of the platform’s operational health.	
Exceptional Flows	
E1 – Statistics service error: If the aggregation query fails, the analytics cards display “N/A” and an error notification is shown to the Admin.	

Table 7.7: Use Case – consult analytics

7.1.4.3 Transaction Finalization

Use Case Name	Request transaction approval
Actor(s)	Sub-Agent
Pre-conditions	The Sub-Agent is authenticated. The target property is in “Available” status and is assigned to the Sub-Agent. At least one completed visit for this property exists.
Post-conditions	A transaction request record is created with status “Pending”, awaiting Head-Agent review. The property status is temporarily flagged as “Pending Finalization”.
Nominal Flow	
1. The Sub-Agent navigates to the property in their assigned listings view on the Agent dashboard. 2. The Sub-Agent clicks the “Request Sale” or “Request Rent” action button for the target property. 3. A confirmation dialog appears summarizing the finalization type and asking for confirmation. 4. Upon confirmation, the frontend calls <code>POST /reports/request</code> with the property ID and finalization type. 5. The backend creates a new <code>TransactionRequest</code> record with status “Pending” and notifies the Head-Agent via email. 6. The property’s status badge updates on the Sub-Agent’s dashboard to “Pending Finalization”.	
Exceptional Flows	
E1 – Property not assigned to Sub-Agent: The backend returns a 403 Forbidden error, preventing the unauthorized request. E2 – Pending request already exists: The system informs the Sub-Agent that a request for this property is already awaiting Head-Agent review.	

Table 7.8: Use Case – Request transaction approval

Use Case Name	Approve transaction
Actor(s)	Head-Agent
Pre-conditions	A TransactionRequest with status “Pending” exists and is linked to a property managed by the Head-Agent. The Head-Agent has received the email notification from the Sub-Agent’s request.
Post-conditions	The transaction request status changes to “Approved”. The property’s availability status is updated to “Approved Sold” or “Approved Rented”. The Sub-Agent can now see the updated status on their dashboard and proceed to complete the transaction. No email is sent at this stage; email notifications are triggered only upon transaction completion or refusal.
Nominal Flow	
1. The Head-Agent accesses the “Pending Requests” section of the Agency dashboard, having received an email notifying them of the Sub-Agent’s transaction request. 2. The Head-Agent reviews the transaction request details (property title, Sub-Agent name, finalization type: Sale or Rent). 3. The Head-Agent clicks the “Approve” button for the selected request. 4. The frontend calls the backend approval endpoint with the request ID. 5. The backend updates the TransactionRequest status from “Pending” to “Approved”. 6. The backend simultaneously updates the property’s availability status to “Approved Sold” or “Approved Rented”. 7. The request status is reflected on the Head-Agent’s dashboard. The Sub-Agent, upon refreshing their dashboard, can see that the request has been approved and a “Complete” action button becomes available.	
Exceptional Flows	
E1 – Request already processed: The system returns an error if the request is no longer in “Pending” state (e.g., it was already approved or refused), and the Head-Agent is notified accordingly. E2 – Unauthorized access: The backend returns a 403 Forbidden error if the property does not belong to the requesting Head-Agent.	

Table 7.9: Use Case – Approve transaction

Use Case Name	Complete transaction
Actor(s)	Sub-Agent
Pre-conditions	A TransactionRequest with status “Approved” exists and is visible on the Sub-Agent’s dashboard. The property is in “Approved Sold” or “Approved Rented” status, granting the Sub-Agent permission to finalize the deal.
Post-conditions	The property’s status is permanently updated to “Sold” or “Rented”. The transaction request status changes to “Completed”. The Sub-Agent and Head-Agent are notified via email that the transaction has been completed. The Admin receives a confirmation email with an attached PDF transaction report. The Client receives a confirmation email notifying them of the finalized property sale or rental.
Nominal Flow	
1. The Sub-Agent accesses the Agent dashboard and locates the transaction request that has been approved by the Head-Agent (status: “Approved”). 2. The Sub-Agent clicks the “Complete” action button on the approved request row. 3. A confirmation dialog appears summarizing the finalization details (property, type: Sale/Rent). 4. Upon confirmation, the frontend calls <code>POST /properties/{property_id}/finalize-transaction</code> with the action “complete”. 5. The backend updates the TransactionRequest status to “Completed”. 6. The backend updates the property status to “Sold” or “Rented”, and sets the buyer/renter ID. 7. The backend updates the associated visit status to “Finished” and automatically cancels any other pending visits linked to this property. 8. The email service asynchronously dispatches notifications: a. The Sub-Agent and Head-Agent receive an email confirming the successful transaction completion. b. The Admin receives a comprehensive email with an attached PDF transaction report. c. The Client receives an email confirming the finalization of their property sale or rental. 9. The deal is archived in the finalized history and becomes visible in the Head-Agent’s “Consult Validated Properties” section.	
Exceptional Flows	
E1 – Request not in Approved state: The backend returns a 400 error if the request status is not “Approved”, preventing premature completion. E2 – Email service failure: The database update succeeds and the transaction is finalized; however, the email delivery failure is logged server-side and flagged for automated retry.	

Table 7.10: Use Case – Complete transaction

Use Case Name	Receive email request for transaction
Actor(s)	Head-Agent
Pre-conditions	A Sub-Agent has submitted a transaction request (Request Sale or Request Rent) for a property managed by the Head-Agent. The SMTP email service is operational.
Post-conditions	The Head-Agent receives an email summarizing the transaction request details, prompting them to review and act on it via the Agency dashboard.
Nominal Flow	
1. A Sub-Agent submits a Sale or Rent transaction request via the Agent dashboard. 2. The backend (<code>backend/routers/reports.py</code>) creates the Transaction-Request record and immediately triggers the email service. 3. The email service (<code>backend/services/email.py</code>) composes a notification email addressed to the Head-Agent containing: the property title, the Sub-Agent's name, and the requested transaction type (Sale or Rent). 4. The email is dispatched via SMTP. 5. The Head-Agent receives the email in their inbox and is prompted to navigate to the Agency dashboard to review and act on the pending request.	
Exceptional Flows	
E1 – SMTP service unavailable: The email delivery fails. The TransactionRequest is still persisted in the database and remains visible on the Agency dashboard. The failure is logged server-side for later investigation.	

Table 7.11: Use Case – Receive email request for transaction

Use Case Name	Receive email response for transaction request
Actor(s)	Sub-Agent
Pre-conditions	The Head-Agent has made a decision on a pending TransactionRequest: either completing (which the Sub-Agent acts on) or refusing it. The SMTP email service is operational.
Post-conditions	The Sub-Agent receives a role-specific email reflecting the outcome: a completion confirmation (when the Sub-Agent themselves completes the transaction) or a refusal notification (when the Head-Agent refuses the request).
Nominal Flow	
1a. (Completion path) The Sub-Agent completes an approved transaction via their dashboard. The backend triggers the email service to send a completion confirmation email to the Sub-Agent, the Head-Agent, the Admin (with PDF report), and the Client. 1b. (Refusal path) The Head-Agent refuses a pending transaction request via their dashboard. 2b. The backend calls <code>backend/services/email.py</code> to send a refusal notification email to the Sub-Agent. 3b. The email contains the reason for refusal (if provided by the Head-Agent) and advises the Sub-Agent to discuss with the Head-Agent or resubmit the request if appropriate. 4. The Sub-Agent receives the email in their inbox and can take corrective action or acknowledge the outcome.	
Exceptional Flows	
E1 – SMTP service unavailable: The email delivery fails. The transaction outcome (completion or refusal) is still persisted in the database. The failure is logged and flagged for retry.	

Table 7.12: Use Case – Receive email response for transaction request

Use Case Name	Receive transaction report email
Actor(s)	Admin
Pre-conditions	A Sub-Agent has completed a transaction (status: “Completed”). The SMTP email service is operational and the Admin’s email address is configured in the system.
Post-conditions	The Admin receives an email containing a PDF transaction report summarizing the finalized deal (property, parties involved, transaction type, date).
Nominal Flow	
1. A Sub-Agent completes an approved transaction via the Agent dashboard. 2. The backend finalizes the transaction in the database and immediately calls the email service (<code>backend/services/email.py</code>). 3. The email service generates a formatted PDF transaction report server-side, containing: property title, type (Sale/Rent), finalization date, the Sub-Agent’s name, the Head-Agent’s name, and the Client’s contact information. 4. The PDF is attached to a summary email and dispatched to the Admin’s configured email address via SMTP. 5. The Admin receives the email and can open or archive the PDF report for compliance and record-keeping purposes.	
Exceptional Flows	
E1 – PDF generation failure: If the server-side PDF generation fails, the transaction is still recorded in the database. The system logs the error and the Admin can still download the report manually from the Reports dashboard. E2 – SMTP delivery failure: The email with the report fails to deliver. The failure is logged and flagged for retry; the transaction data remains intact and accessible.	

Table 7.13: Use Case – Receive transaction report email

Use Case Name	Refuse transaction
Actor(s)	Head-Agent
Pre-conditions	A TransactionRequest with status “Pending” exists and is linked to a property managed by the Head-Agent.
Post-conditions	The TransactionRequest status is updated to “Refused”. The property’s status reverts to “Available”. The Sub-Agent is notified via email.
Nominal Flow	
1. The Head-Agent reviews the pending request and decides to refuse it. 2. The Head-Agent clicks the “Refuse” button and optionally enters a reason for the refusal. 3. The frontend calls <code>POST /reports/refuse/{request_id}</code> with the optional refusal reason. 4. The backend updates the TransactionRequest status to “Refused” and restores the property status to “Available”. 5. The email service sends a refusal notification email to the Sub-Agent including the stated reason. 6. The request is removed from the pending list and logged in the finalization history with “Refused” status.	
Exceptional Flows	
E1 – Request already finalized: The system informs the Head-Agent that the request can no longer be modified. E2 – Email delivery failure: The refusal is recorded in the database, but the email delivery failure is flagged for retry.	

Table 7.14: Use Case – Refuse transaction

Use Case Name	consult validated properties
Actor(s)	Head-Agent
Pre-conditions	The Head-Agent is authenticated and at least one transaction has been finalized (Sold or Rented) under their portfolio.
Post-conditions	A chronological list of all finalized (approved or refused) transaction requests is displayed with their outcomes.
Nominal Flow	
1. The Head-Agent accesses the “Finalized Deals” section of the Agency dashboard. 2. The frontend calls <code>GET /reports/history</code> with the Head-Agent’s token. 3. The backend retrieves all <code>TransactionRequest</code> records associated with the Head-Agent’s properties, regardless of outcome. 4. The results are displayed in a table showing: property title, Sub-Agent name, finalization type, outcome (Approved/Refused), and date. 5. The Head-Agent can sort and filter the history by status or date range for quick reference.	
Exceptional Flows	
E1 – No finalized deals: The table displays an empty state indicating that no finalized transactions exist yet.	

Table 7.15: Use Case – consult validated properties

7.1.5 Detailed Sequence Diagrams

This section presents the dynamic interaction flows for the notification triggers, dashboard data aggregations, and the transaction finalization lifecycle implemented in Sprint 4.

7.1.5.1 Sequence Diagram “Automated Visit Reminder”

This sequence illustrates the Meeting Reminder Service n8n workflow. Operating on a CRON schedule, the workflow calls the FastAPI endpoint to fetch upcoming visits, then dispatches parallel notifications via the Telegram Bot API (to the Client) and SMTP (to the Sub-Agent), completing without any manual intervention.

[Placeholder: Insert Automated Visit Reminder Sequence Diagram]

Figure 7.3: Sequence Diagram “Automated Visit Reminder (n8n)”

7.1.5.2 Sequence Diagram “Approve Transaction Request”

This sequence illustrates the Head-Agent approval workflow. After the Head-Agent clicks “Approve”, the backend updates the TransactionRequest status to “approved” and the Property status to “approved_sold” or “approved_rent”, allowing the Sub-Agent to proceed with the completion.

[Placeholder: Insert Approve Transaction Request Sequence Diagram]

Figure 7.4: Sequence Diagram “Approve Transaction Request”

7.1.5.3 Sequence Diagram “Complete Transaction”

This sequence illustrates the final completion workflow executed by the Sub-Agent. Upon clicking “Complete” for an approved transaction, the backend updates the TransactionRequest status to “completed”, changes the Property status to “sold” or “rented”, cancels any other active scheduled visits for that property, and automatically triggers the email service to generate and dispatch confirmation notifications along with the PDF report to the Admin and Client.

[Placeholder: Insert Complete Transaction Sequence Diagram]

Figure 7.5: Sequence Diagram “Complete Transaction”

7.1.5.4 Sequence Diagram “Refuse Transaction Request”

This sequence shows the refusal workflow. The backend reverts the property status to “Available”, marks the request as “Refused”, and triggers a targeted email notification to the Sub-Agent containing the Head-Agent’s stated reason for refusal.

[Placeholder: Insert Refuse Transaction Request Sequence Diagram]

Figure 7.6: Sequence Diagram “Refuse Transaction Request”

7.1.5.5 Sequence Diagram “Consult Team Calendar”

This sequence shows the Head-Agent loading the team calendar. The backend runs a JOIN query across the visits, users, and properties tables to aggregate all Sub-Agent appointments, which the frontend maps onto the interactive calendar component with real-time filter support.

[Placeholder: Insert Team Calendar Sequence Diagram]

Figure 7.7: Sequence Diagram “Consult Team Calendar”

7.2 Implementation

This section presents the data visualization and management interfaces developed during Sprint 4, the final sprint of the Elite Estate platform.

7.2.1 Automated Notification System

The Meeting Reminder Service operates entirely in the background via n8n. Administrators can view the workflow execution logs directly in the n8n interface to confirm that reminder batches were dispatched successfully. The Client receives the TAIA Telegram reminder message and the Sub-Agent receives the formatted email, each containing the visit’s property name, address, date, and time.

[Placeholder: Insert Screenshot of a TAIA Telegram Reminder Message]

Figure 7.8: TAIA Automated Visit Reminder via Telegram

[Placeholder: Insert Screenshot of the Reminder Email received by Sub-Agent]

Figure 7.9: Automated Visit Reminder Email to Sub-Agent

7.2.2 Head-Agent Calendars and Analytics Dashboard

The Agency dashboard provides two distinct calendar views: the Team Calendar (aggregating all Sub-Agent appointments for supervisory oversight) and the Personal Calendar (displaying the Head-Agent's own schedule). analytics summary cards present total listings, team visit counts, and completed transaction volumes. A bar chart visualizes individual Sub-Agent performance comparisons.

[Placeholder: Insert Screenshot of the Head-Agent Team Calendar]

Figure 7.10: Head-Agent Team Calendar Dashboard

[Placeholder: Insert Screenshot of the Head-Agent Analytics Statistics Dashboard]

Figure 7.11: Head-Agent Analytics Statistics and Team Performance Dashboard

7.2.3 Transaction Finalization Interface

The “Pending Requests” section of the Agency dashboard presents all transaction requests submitted by the Head-Agent’s Sub-Agents. Each row contains the property title, requesting agent name, finalization type (Sale/Rent), and action buttons (Approve / Refuse). Once approved by the Head-Agent, the request enters the “Approved” state, enabling the Sub-Agent to complete the transaction on their dashboard. The “Finalized Deals” history tab provides an auditable chronological log of all past finalization decisions after they are completed or cancelled by the Sub-Agent.

[Placeholder: Insert Screenshot of the Pending Transaction Requests Panel]

Figure 7.12: Pending Transaction Requests Panel (Head-Agent)

[Placeholder: Insert Screenshot of the Finalized Deals History Table]

Figure 7.13: Finalized Deals History

7.2.4 Admin Global Analytics Dashboard

The Admin dashboard provides a global view of platform health with four primary analytics widgets (Total Users by Role, Total Property Listings, Total Visits, Total Finalized Transactions) and a trend line chart visualizing platform growth. The transaction report download button on this page allows the Admin to export a comprehensive PDF record of all finalized deals.

[Placeholder: Insert Screenshot of the Admin Global Analytics Dashboard]

Figure 7.14: Admin Global Analytics and Statistics Dashboard

Conclusion

Sprint 4 completed the full development lifecycle of the Elite Estate platform. The automated notification system ensures that no visit is missed through timely TAIA and email reminders, while the structured finalization workflow establishes a clear, auditable chain of approval for every property transaction. The comprehensive analytics dashboards empower both Head-Agents and Administrators with real-time operational insights, making the Elite Estate platform a complete, production-ready real estate automation solution.